FIL · AI FOUNDATIONS FOR ENGINEERS

LLM App & Agent Development

From a single API call to production-grade AI agents in a regulated enterprise

LLM fundamentals | AI Gateway | Prompts & context | Tools · MCP · Memory | ReAct agents | Guardrails & evals

LEARNING OUTCOMES

What you will be able to do by the end of this session

01

Explain the engine

Describe how an LLM turns tokens into text and how temperature, top-p and max tokens change its behaviour.

02

Call LLMs the enterprise way

Route every request through the AI Gateway, read the response object correctly, and choose streaming when UX demands it.

03

Build reliable LLM apps

Use prompt templates and conversation history deliberately, knowing the model itself is stateless.

04

Design AI agents

Combine an LLM with tools, MCP, memory, workflow patterns and skills using the ReAct loop.

05

Ship responsibly

Apply guardrails, evaluation and observability before anything reaches a customer or colleague.

Each outcome maps to one part of the session. Every concept is paired with a diagram, and the key ones with runnable code.

AGENDA

The roadmap: five parts, each building on the last

01

How an LLM generates text

Tokens, next-token prediction, temperature and sampling parameters

02

Calling LLMs the enterprise way

AI Gateway, request lifecycle, response anatomy, streaming

03

Building LLM applications

Prompt templates, anatomy of a prompt, multi-turn context

04

From LLM apps to AI agents

Agent anatomy, ReAct, tools & MCP, memory, patterns, skills

05

Production readiness

Guardrails, evaluation, observability and a go-live checklist



The logic: understand the engine → call it safely → shape its input → give it tools, memory and a loop → make it trustworthy. Then practise it: six labs and a Friday capstone.

CONTEXT

AI at Fidelity International

What every beginner on an FIL AI team should know: who FIL is, what it has shipped, the stack it hires for, the controls it expects and the rules it builds within.

CONTEXT · AI AT FIDELITY INTERNATIONAL

First, know which Fidelity you work for

1969

founded as FMR's international arm; independent since 1980

US$1.09tn

client assets (31 Dec 2025)

4,500+

people in India: Gurugram, Bengaluru, Mumbai

2001

India site opened; now FIL's largest capability hub

Two separate companies

Fidelity International (FIL) serves clients in Europe, Asia and other non-US markets. Fidelity Investments (FMR) is a different, Boston-based company serving the US.

Why this matters on day one

Most “Fidelity AI” news, blogs and job posts online are about Fidelity Investments. Their tools and platforms are not FIL’s. Check the source before you assume.

Regulated, multi-jurisdiction

FIL is an asset manager operating across the UK, EU and Asia, so every AI system is built inside financial regulation, not just security policy.

Sources: Wikipedia, “Fidelity International” (assets, history); FIL Careers, “Working for Fidelity in India”. Public sources, Oct 2026.

CONTEXT · AI AT FIDELITY INTERNATIONAL

FIL's AI in production: Freya, a conversational assistant for investors



Freya

Launched
September 2025, phased roll-out

Users
UK Personal Investing platform, about 500,000 customers

What it is
A generative AI conversational interface that reasons through answers instead of following scripts

Next
Planned for Workplace Investing and Fidelity Adviser Solutions

Design choices worth copying



Information, not advice Part 5 · Capstone
It will not give financial advice or personal recommendations: the same rule as capstone FR-6.



Boundary detection Part 5 · Lab 6
Safety checks detect and halt conversations that approach advice or regulatory boundaries.



Human oversight and a way out Part 4 · ReAct
Built with human oversight; customers can switch to client services for complex needs.



Phased roll-out Part 5 · Evals
One platform first, then extend: ship small, measure, widen.

Sources: FStech and IFA Magazine, Sept 2025 (quoting Stuart Warner, Head of Global Platform Solutions, FIL). The underlying model provider has not been disclosed publicly.

CONTEXT · AI AT FIDELITY INTERNATIONAL

The AI stack in FIL : what to learn first

Layer	Named in FIL's public AI job postings (2026)	Where in this course
Model access	Azure OpenAI, AWS Bedrock, OpenAI, Anthropic, open-source models (a multi-model strategy)	Part 2 · AI Gateway
Agents & tools	LangGraph, LangChain, Semantic Kernel, CrewAI, AutoGen · Model Context Protocol (MCP)	Part 4 · Labs 4–5
RAG & data	Chunking, hybrid retrieval, reranking, citations · pgvector, OpenSearch, Azure AI Search, Snowflake	Part 4 · Memory
Evaluation	DeepEval, RAGAS, LangSmith, Phoenix, Promptfoo, TruLens · faithfulness, relevancy, hallucination	Part 5 · Lab 6
AI-assisted coding	GitHub Copilot, Claude Code, Cursor · standards for AI-generated code	Capstone rules
Engineering base	Python + FastAPI, TypeScript, Docker, Kubernetes, IaC, CI/CD, DevSecOps	Prior FIL modules



Read this as "skills FIL asks for", not a confirmed inventory: postings say "or equivalent". Your internal platform catalogue lists what is actually approved.

Sources: FIL job postings for AI Engineer and AI Solutions Engineer (Asset Management Platforms, Bengaluru/Gurgaon/Dublin) and AI Test Engineer (Bengaluru), 2026.

CONTEXT · AI AT FIDELITY INTERNATIONAL

The controls FIL expects, and where you practise them

Prompt injection & jailbreaks
Untrusted text never becomes instructions
Lab 6 · capstone AT-04

Sensitive-data exposure
Redaction, approved models, no data leakage
Lab 6 · capstone NFR-5

Hallucination & unsafe output
Grounding, citations, content filtering
Part 5 · FR-5

Unauthorised tool use
Least privilege, validated arguments
Lab 4 · NFR-6

Human-in-the-loop
Approval before irreversible actions
Lab 6 · FR-8

Evaluation datasets & quality gates
Automated tests in CI on every change
Lab 6 · run_evals

Prompt & model config management
Versioned prompts, pinned model settings
Lab 3 · AR-8

Third-party model risk
Approved providers via the gateway only
Part 2 · AR-1

Sources: FIL AI Solutions Engineer and AI Test Engineer job postings, 2026 (controls and practices quoted from the role descriptions).

CONTEXT · AI AT FIDELITY INTERNATIONAL

The regulatory frame you build inside

UK · FCA

- No new AI-specific rules: the FCA relies on existing frameworks (Sept 2025).
- Consumer Duty: AI that touches customers must deliver good outcomes.
- SM&CR: a named senior manager is accountable, so decisions must be explainable and auditable.
- FCA AI Lab and live testing exist to support firms.

EU · AI Act

- In force, with obligations phased in.
- Digital Omnibus, Regulation (EU) 2026/1744, moved the deadlines.
- Annex III high-risk systems: from 2 Dec 2027.
- Annex I high-risk systems: from 2 Aug 2028.
- Watermarking AI-generated content (Art. 50(2)), systems already on the market: from 2 Dec 2026.

Everywhere else

- FIL operates across Asia-Pacific too: local regulators and data rules apply.
- Data protection applies to prompts, logs, memory and traces.
- Rules change fast: Compliance and Legal own the interpretation, not engineers.



Engineer’s takeaway: design for auditability from day one. Traces, versioned prompts, human approval and evals are how you prove compliance later.

Sources: A&O Shearman on the FCA's AI approach (9 Sept 2025); Orrick (29 Jul 2026) and Gibson Dunn on the EU AI Act Digital Omnibus. Not legal advice.

CONTEXT · AI AT FIDELITY INTERNATIONAL

Sources for this section

Fidelity International, Wikipedia History, ownership, client assets

FIL Careers: Working for Fidelity in India India sites, headcount, capability hub

FStech: Fidelity launches conversational genAI tool (Sept 2025) Freya launch details

IFA Magazine: Fidelity Personal Investing launches Freya Freya safeguards; Stuart Warner quotes

FIL job posting: AI Solutions Engineer (AMP, Bengaluru/Gurgaon) Stack, controls, practices

FIL job posting: AI Test Engineer (Bengaluru) Evaluation tools and metrics

A&O Shearman: FCA approach to AI (Sept 2025) No AI-specific rules; Consumer Duty, SM&CR

Orrick: EU AI Act Digital Omnibus finalised (Jul 2026) New high-risk deadlines

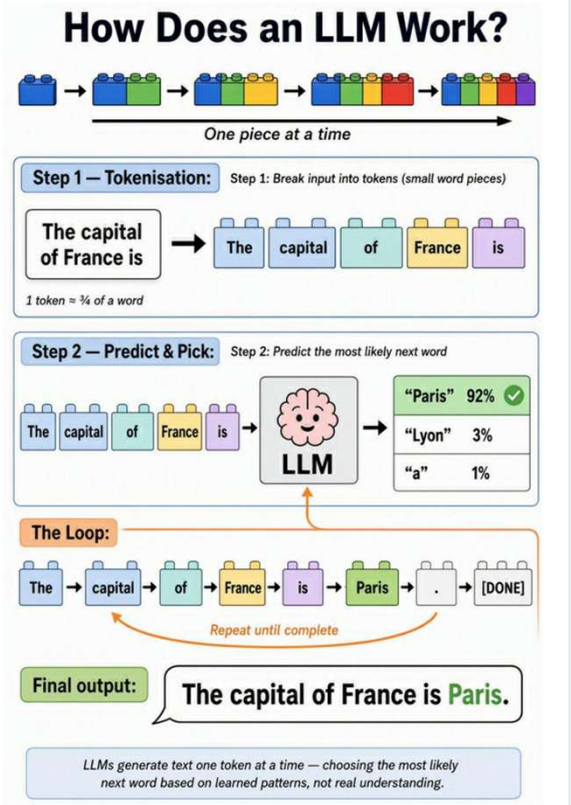
Compiled from public sources in October 2026. Job postings describe skills sought, not confirmed deployments. Nothing here is legal advice.

PART 01

How an LLM generates text

Before writing any code, understand the engine: an LLM predicts one token at a time, and a few parameters control how it picks.

An LLM writes text one token at a time



1

Tokenise the input

Text is split into tokens: sub-word pieces. In English, one token is roughly ¾ of a word, so 1,000 tokens ≈ 750 words. Cost, latency and context limits are all measured in tokens, not characters.

2

Predict a probability distribution

The model scores every token in its vocabulary given everything so far. For "The capital of France is", "Paris" might get 92%, "Lyon" 3%. A sampling strategy then picks one.

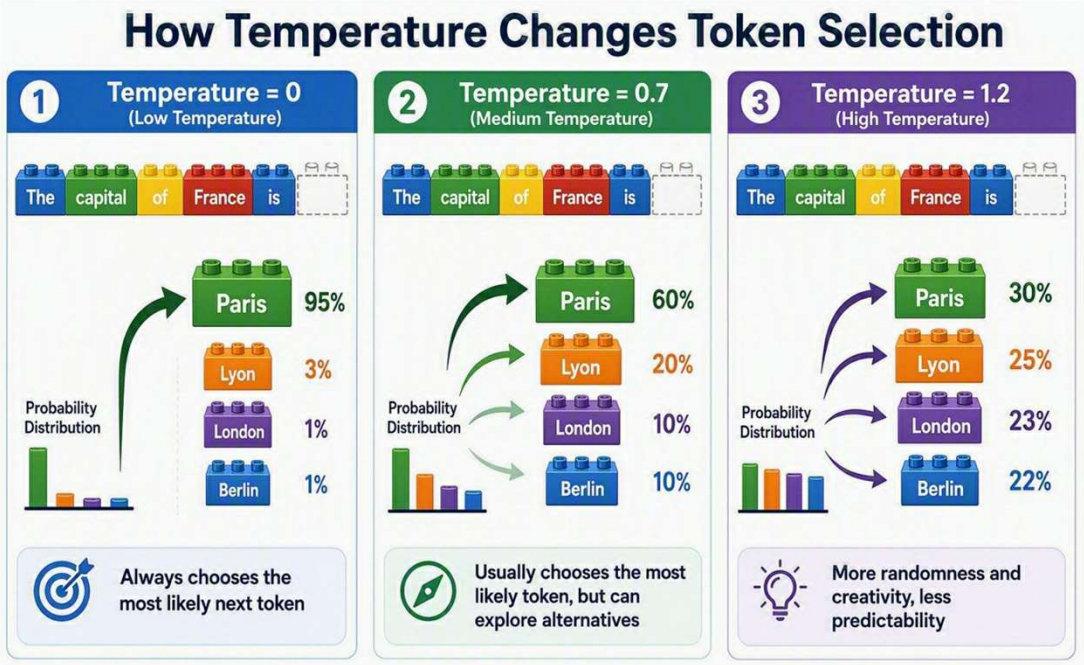
3

Append and loop

The chosen token is added to the input and the model runs again, until it emits a stop token or hits the max-token limit. A 500-word answer is roughly 650 forward passes.

Engineering consequence: the model has no plan and no lookup table of facts. It continues patterns, so output quality depends on what you put in the context.

Temperature reshapes the probability curve before a token is picked



Temperature ≈ 0 · near-deterministic

The distribution is sharpened so the top token almost always wins. Use for extraction, classification, code and anything that must be repeatable.

Temperature 0.5 – 0.8 · balanced

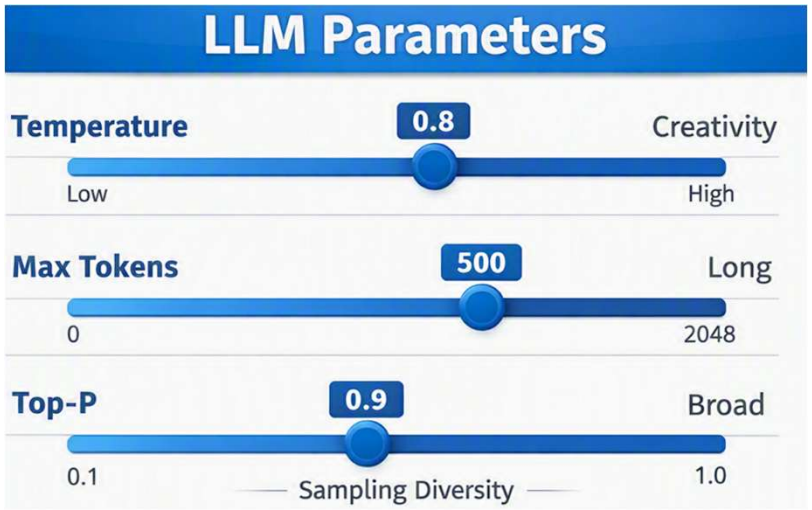
Mostly picks the favourite but sometimes explores alternatives. A sensible default for chat, summaries and drafting.

Temperature > 1 · creative, riskier

The curve flattens and unlikely tokens get real chances. Useful for brainstorming; higher risk of off-topic or wrong output.

Even at temperature 0, outputs can differ between runs. Never rely on it alone for reproducibility; log prompts, settings and model versions.

The sampling parameters you will set on almost every call



Parameter	What it controls	Typical setting
`temperature`	Randomness of token choice. Lower = focused, higher = varied.	0–0.2 for extraction and code; 0.7 for chat
`top_p`	Nucleus sampling: only tokens inside the top p of cumulative probability can be picked.	Leave at 1.0, or tune either it or temperature, not both
`max_tokens`	Hard ceiling on output length. Caps cost and latency.	Size to the expected answer plus headroom
`stop`	Strings that end generation early.	Use for structured or delimited outputs
`seed`	Best-effort reproducibility where the provider supports it.	Set in tests and evaluations

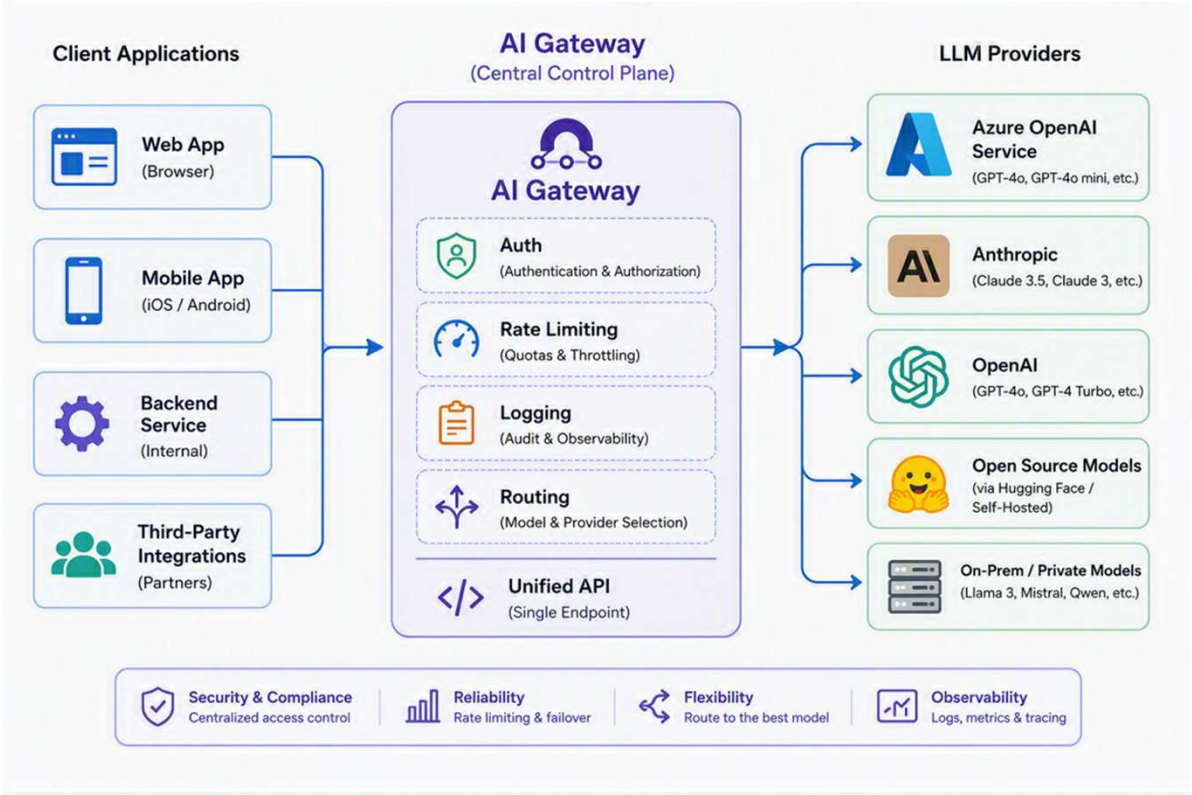
Watch finish_reason: if it returns "length", the answer was cut off by max_tokens. Treat that as an error to handle, not a valid answer.

PART 02

Calling LLMs the enterprise way

In an enterprise, applications never call model vendors directly. Every request goes through one governed control plane: the AI Gateway.

Why every call goes through the AI Gateway



One endpoint, many providers

Apps speak one OpenAI-compatible API. The gateway routes to Azure OpenAI, Anthropic, open-source or on-prem models, so swapping a model is configuration, not code.

Security & compliance by default

Central authentication, no vendor keys in app code, and approved models only, which is essential in regulated financial services.

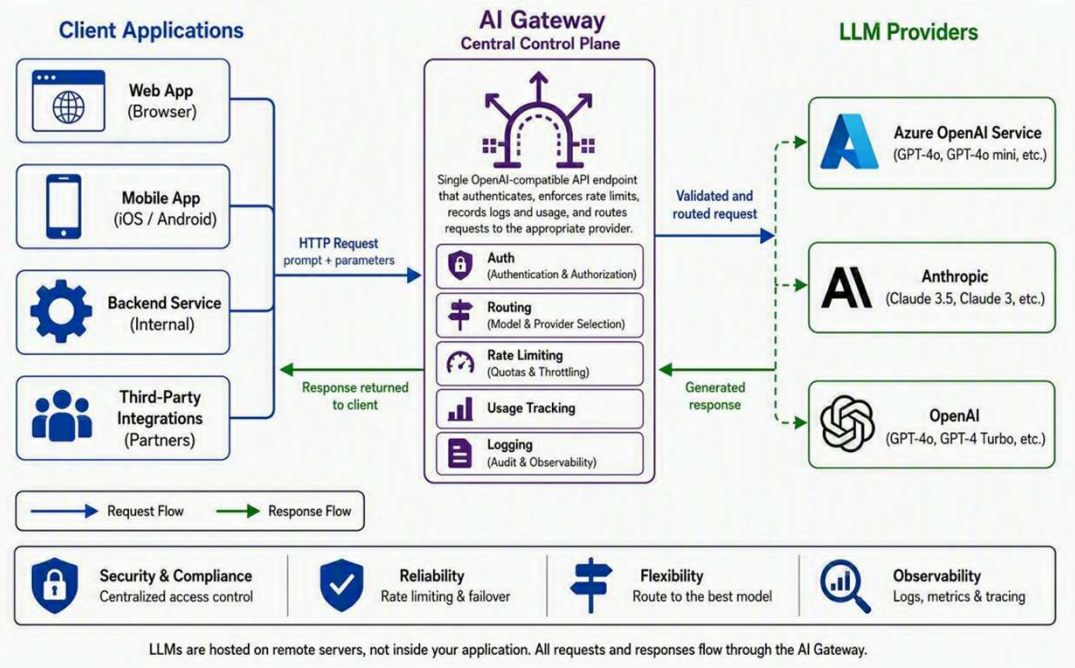
Cost and reliability control

Quotas, rate limits and failover are enforced once, for everyone.

Full auditability

Every prompt, response and token count is logged for audit, troubleshooting and chargeback.

The life of a request: from your app to a model and back



1. **Send**
   App sends an HTTP request: messages plus parameters.
2. **Authenticate**
   Gateway checks identity and entitlement.
3. **Govern**
   Rate limits, quotas and policies are applied.
4. **Route**
   Request is forwarded to the selected provider.
5. **Record**
   Response returns; usage and logs are captured.

LLMs run on remote servers, not inside your app. Design for network latency, timeouts, retries and rate-limit (HTTP 429) responses from day one.

Your first call: the standard SDK pointed at the gateway

python · first_call.py

```python
from openai import OpenAI
import os

# Point the standard SDK at the gateway, not at a vendor
client = OpenAI(
    base_url=os.environ["AI_GATEWAY_URL"],
    api_key=os.environ["AI_GATEWAY_KEY"],   # never hard-code
)

resp = client.chat.completions.create(
    model="gpt-4o-mini",       # gateway routes by model name
    messages=[
        {"role": "system", "content": "Be concise."},
        {"role": "user", "content": "What is an API?"},
    ],
    temperature=0.2,
    max_tokens=200,
    timeout=30,
)

print(resp.choices[0].message.content)
print(resp.usage.total_tokens)
```

OpenAI-compatible = portable

Because the gateway speaks the OpenAI wire format, the same SDK works whichever provider sits behind it.

Secrets live in the environment

Keys come from a vault or environment variables, never source control or notebooks.

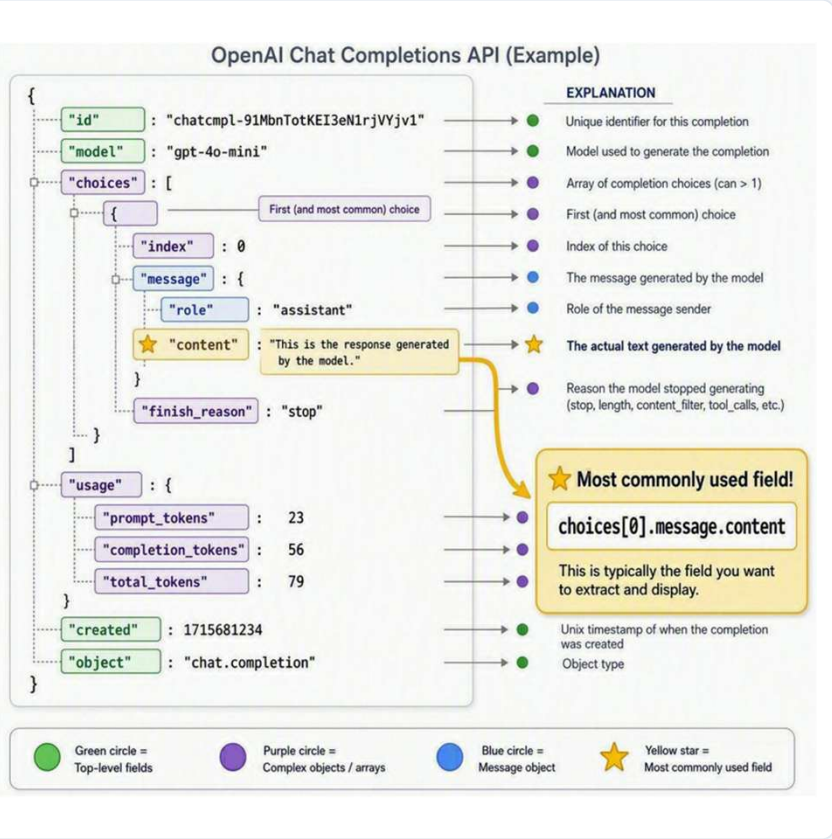
Roles structure the conversation

system sets behaviour and rules; user carries the request; assistant holds prior model replies.

Always read usage

Token counts drive cost dashboards and catch runaway prompts early.

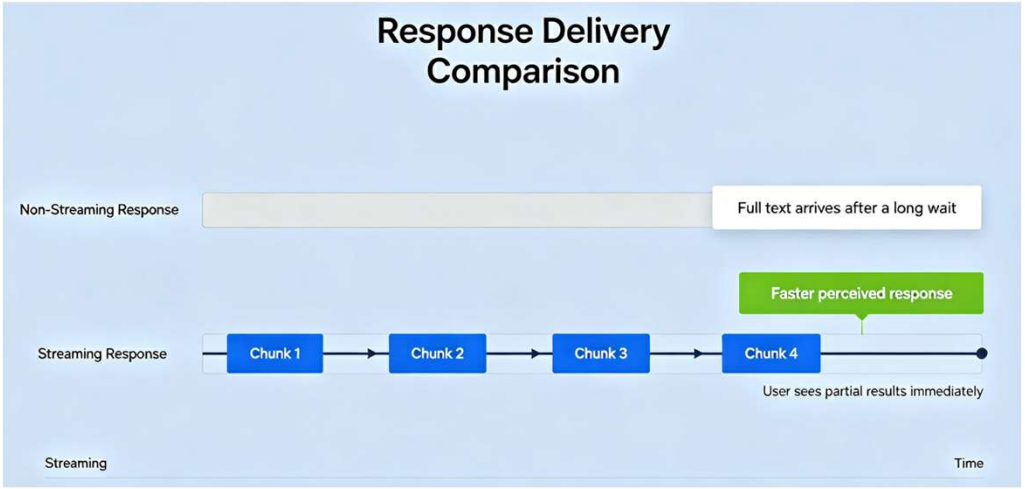
Reading the response: know which fields matter and why



Field	Why you care
`choices[0].message.content`	The generated text: what you display or parse
`choices[0].finish_reason`	stop = complete · length = truncated · tool_calls = model wants a tool · content_filter = blocked
`choices[0].message.tool_calls`	Present when the model asks you to run a tool (Part 4)
`usage.prompt / completion tokens`	Cost, quota tracking and context-window monitoring
`id · model · created`	Correlation IDs for logs, audit and support tickets

Production rule: check finish_reason before trusting content. A truncated or filtered answer must be handled, not displayed.

Streaming: same total time, far better perceived speed



	Non-streaming	Streaming
First visible text	After the full answer	Within the first second
Best for	Batch jobs, JSON parsing, back-end steps	Chat UIs, long answers
Complexity	Simple: one response object	Handle chunks, partial output and errors mid-stream

python · streaming

```
stream = client.chat.completions.create(
    model=MODEL, messages=msgs,
    stream=True)
for chunk in stream:
    if not chunk.choices:
        continue
    text = chunk.choices[0].delta.content
    if text:
        print(text, end="", flush=True)
```

Key metric: time to first token

Users judge speed by when text starts appearing, not when it finishes.

Guardrail caveat

Output checks run on partial text. Buffer or post-validate for high-risk content.

FILXGPT APIS · AUTHENTICATION

How your app gets an access token

The client-credentials flow with MSAL for Python, explained step by step

client_id + client_credential + tenant_id → access_token

WHY A TOKEN?

No token, no call: FILxGPT only answers callers who prove who they are

1 · YOU

Your app

A Python script or service holding the three values from the SRM ticket



2 · VERIFIER

Entra ID

Microsoft's identity service (Azure AD) checks the app and issues a signed token



3 · DESTINATION

FILxGPT API

Accepts a request only when it carries a valid Bearer token

The token is short-lived proof of identity, scoped to one API. You never send your secret to FILxGPT; you send the token.

THE INPUTS

Three values, all from your SRM ticket

Parameter	What it is	Think of it as
client_id	The app's registered identity (Application ID). Not secret on its own.	Your username
client_credential	The client secret that proves the request really comes from your app.	Your password
tenant_id	FIL's Azure AD directory, where your app is registered.	Which organisation

Treat client_credential like a password: never paste it into code you commit, chat, or share.

OAUTH 2.0 CLIENT-CREDENTIALS FLOW

Four steps, and no user ever logs in

1

Point to the tenant

Build the authority: Microsoft's login address plus FIL's tenant_id.

2

Build the client

Create a confidential client from client_id, secret and authority.

3

Ask for a token

Request a token for the FILxGPT scope. Entra ID returns it.

4

Call the API

Send it in the header: Authorization: Bearer <token>

"Client credentials" means the app signs in as itself. That is why it suits back-end services, scripts and agents.

THE CODE · STEPS 1 AND 2

Set up the credentials and the client

```
from msal import ConfidentialClientApplication

client_id = 'xxxxxx'
client_credential = 'yyyyyy'
tenant_id = 'zzzzzz'

authority = "https://login.microsoftonline.com/" + tenant_id

client = ConfidentialClientApplication(
        client_id=client_id,
        client_credential=client_credential,
        authority=authority,
)
```

Import. MSAL is Microsoft's auth library. Install with pip install msal.

Credentials. Placeholders for the SRM values. In real code, read them from environment variables.

Authority. Microsoft login plus FIL's tenant: where to ask for the token.

Client. "Confidential" means the app can keep a secret, so it runs server-side, never in a browser.

THE CODE · STEP 3

Request the token and pick it out of the response

```
azure_ad_response = client.acquire_token_for_client(
        ["https://uat.filxgptnp.fil.com/.default"]
)

access_token = azure_ad_response["access_token"]

# a successful response looks like:
# {"token_type": "Bearer", "expires_in": 3599,
# "access_token": "eyJ0eXAiOi..."}
```

for_client. The app signs in as itself; no user, no browser.

Scope. Which API the token is for. The /.default suffix means "all permissions already granted to this app".

Response. A Python dict. The token is a JWT string under "access_token".

Failure. No "access_token" key; read "error" and "error_description" instead.

ENVIRONMENTS

Switching UAT and DEV changes one string: the scope

Environment	Scope to pass	Note
UAT	https://uat.filxgptnp.fil.com/.default	Used in the sample code
DEV	https://dev.filxgptnp.fil.com/.default	For development testing

Stays the same

The import, the client set-up and the authority URL.

Keep it in config

Read the scope from an environment variable so one script runs anywhere.

STEP 4 · USING THE TOKEN

Check the response, then send the token as a Bearer header

```
result = client.acquire_token_for_client(scopes=[os.environ["FILXGPT_SCOPE"]])
if "access_token" not in result:
        raise RuntimeError(result.get("error_description"))

headers = {"Authorization": "Bearer " + result["access_token"]}
resp = requests.post(FILXGPT_ENDPOINT, headers=headers, json=payload)
```

Fail loudly. A clear error beats a KeyError three lines later.

Bearer + space. The header value is the word Bearer, a space, then the token.

401 response? The token is missing, expired, or for the wrong environment.

GOOD PRACTICE

Four habits that keep this safe in real projects

Keep secrets out of code

Use environment variables or a vault. Never commit client_credential to Git.

Reuse one client

Create it once at start-up. MSAL keeps the token in memory and reuses it until near expiry.

Expect expiry

Tokens are short-lived (see expires_in). Call acquire again instead of saving tokens to disk.

Never log tokens

A token is a key while it lives. Mask it in logs, errors and screenshots.

RECAP

Three secrets in, one token out, Bearer on every call.

In

client_id, client_credential and tenant_id from SRM

Out

A short-lived access token for the UAT or DEV scope

Use

Authorization: Bearer header on every FILxGPT request

FILXGPT APIS · PART 2

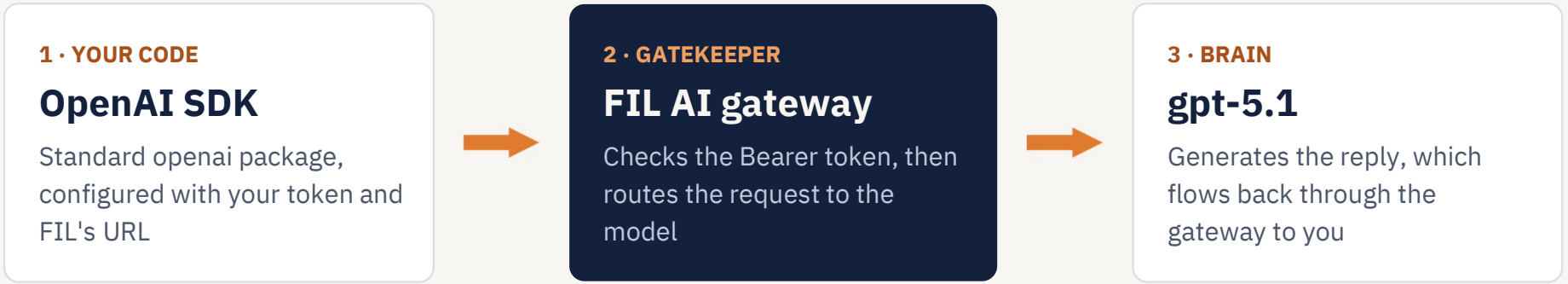
Your first chat call through the FIL AI gateway

Using the OpenAI Python SDK with your access token, explained line by line

access_token → OpenAI client → gpt-5.1 → reply

THE BIG PICTURE

Familiar OpenAI code, but every request goes through FIL's gateway



Your code never talks to the model provider directly. The gateway is where FIL applies access control, routing and governance.

THE CLIENT

Two settings turn the OpenAI SDK into a FILxGPT client

Argument	What you pass	What it does
api_key	access_token from Part 1	Sent on every request as Authorization: Bearer <token>
base_url	FIL gateway URL ending in /llmproxy/v1	Replaces the default api.openai.com address

The "API key" here is **not** an OpenAI key. It is your short-lived Entra ID token, so it expires.

READING THE BASE_URL

The address tells you where you are calling

```
https://fil-ai-gateway-uat.k8s.npiek8sinfra1np1.npaws.ukfilcld/llmproxy/v1
```

fil-ai-gateway-uat

The FIL AI gateway service, UAT environment.

Internal host

FIL's internal Kubernetes cluster address where the gateway runs.

/llmproxy/v1

The LLM proxy path that speaks the OpenAI v1 API.

THE CODE · PART 1

Create the client

```
from openai import OpenAI

client = OpenAI(
        api_key=access_token,
        base_url="https://fil-ai-gateway-uat.k8s.npiek8sinfra1np1.npaws.ukfilcld/llmproxy/v1"
)
```

Import. The official OpenAI library: pip install openai. Nothing FIL-specific to install.

api_key. The token string produced by the MSAL code in Part 1.

base_url. Points the SDK at FIL's gateway. Creating the client sends nothing yet.

THE CODE · PART 2

Send the conversation and print the reply

```
completion = client.chat.completions.create(
        model="gpt-5.1",
        messages=[
                {"role": "developer", "content": "You are a helpful assistant."},
                {"role": "user", "content": "Hello!"}
        ]
)

print(completion.choices[0].message)
```

create(). One HTTP POST to /chat/completions; returns once the reply is ready.

model. A model name the gateway offers; here gpt-5.1.

messages. The conversation so far: a list of role and content pairs.

MESSAGE ROLES

Each message says who is speaking

developer

You, the app builder

Standing instructions: persona, rules, format. The newer name for the "system" role.

user

The person asking

The question or task for this turn. Here, simply "Hello!".

assistant

The model's past replies

Not in this sample. Add earlier replies here to continue a conversation.

The model has no memory between calls. To hold a conversation, you resend the whole message list each time.

THE RESPONSE

Where the answer lives inside the result

```
completion
    .model → "gpt-5.1"
    .choices[0]
        .message.role → "assistant"
        .message.content → "Hello! How can I help you today?"
        .finish_reason → "stop"
    .usage → prompt, completion, total tokens

# just the text, not the whole object:
print(completion.choices[0].message.content)
```

choices[0]. choices is a list; [0] is the first, and usually only, answer.

message. The sample prints the whole message object; .content is the text you want.

finish_reason. "stop" means complete; "length" means it hit the token limit.

usage. Token counts, which drive cost and limits.

PART 1 + PART 2 TOGETHER

The whole thing in one script

```
import os
from msal import ConfidentialClientApplication
from openai import OpenAI

# Part 1: get a token
app = ConfidentialClientApplication(
        os.environ["FILXGPT_CLIENT_ID"],
        client_credential=os.environ["FILXGPT_CLIENT_SECRET"],
        authority="https://login.microsoftonline.com/" + os.environ["FILXGPT_TENANT_ID"],
)
token = app.acquire_token_for_client(["https://uat.filxgptnp.fil.com/.default"])
if "access_token" not in token: raise RuntimeError(token.get("error_description"))

# Part 2: call the model through the gateway
client = OpenAI(api_key=token["access_token"], base_url=os.environ["FILXGPT_BASE_URL"])
reply = client.chat.completions.create(model="gpt-5.1", messages=[{"role": "user", "content": "Hello!"}])
print(reply.choices[0].message.content)
```

WHEN IT BREAKS

Four errors you will meet, and what they mean

401 after about an hour

The token expired. The client keeps the old one, so fetch a new token and rebuild the client.

Environment mismatch

A UAT token must call the UAT gateway. Change scope and base_url together.

Connection error

The gateway host is internal. Check you are on FIL's network or VPN.

Model not found

The model name must be one the gateway offers. Check spelling, like gpt-5.1.

RECAP

Token as api_key, gateway as base_url, conversation in, reply out.

Connect

OpenAI client with the Entra ID token and FIL's gateway URL

Ask

chat.completions.create with a model and a list of messages

Read

choices[0].message.content holds the model's answer

FILXGPT APIS · PART 3

Turning text into vectors with embeddings

What embeddings are, how to create them through the FIL gateway, and how they power semantic search and clustering

"The food was delicious..." → [0.0123, -0.0087, ...]

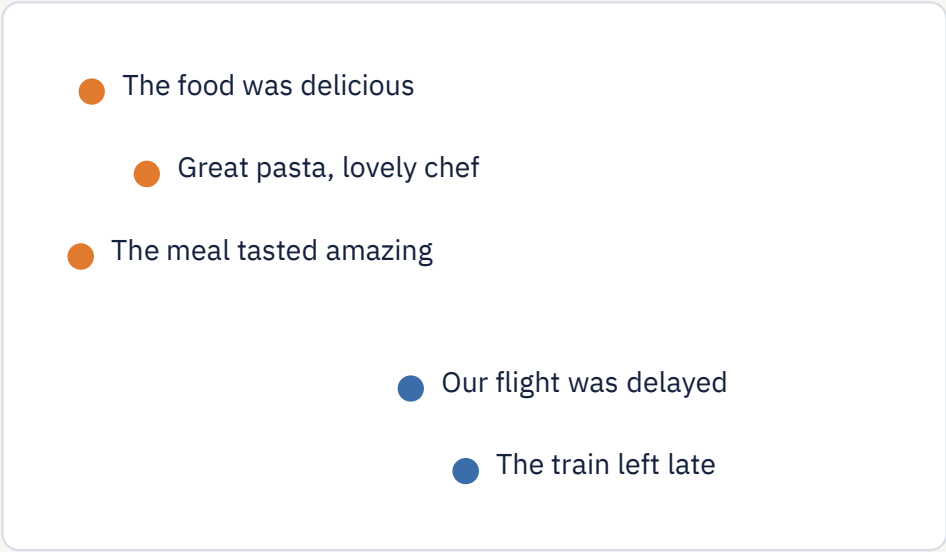
WHAT IS AN EMBEDDING?

Text becomes a point in space, and similar meanings land close together

A vector of numbers. The model reads the text and returns a long list of decimals that encodes its meaning.

Meaning, not words. "Great pasta" and "the meal tasted amazing" share no words, yet sit near each other.

1,536 dimensions. text-embedding-ada-002 returns 1,536 numbers for every input.



A 2D sketch for intuition: food sentences (orange) cluster apart from travel ones (blue).

USAGE EXAMPLES

Two jobs embeddings do well



Semantic search

Search by meaning rather than keywords.

Example: "How do I reset my password?" finds the article titled "Recovering account access".



Clustering

Group similar text for analysis or categorisation.

Example: sort 10,000 customer comments into themes like food, service and delays.

SAME CLIENT, DIFFERENT METHOD

If you can chat, you can already embed

Aspect	Part 2 · Chat	Part 3 · Embeddings
Client	OpenAI(api_key, base_url)	Identical: same token, same gateway
Method	chat.completions.create	embeddings.create
Model	gpt-5.1	text-embedding-ada-002
You send	A list of messages	Text in input
You get	Generated text	A vector of numbers

THE CODE

The sample from the docs, line by line

```
from openai import OpenAI

client = OpenAI(
        api_key=access_token,
        base_url="https://fil-ai-gateway-uat.k8s.npiek8sinfra1np1.npaws.ukfilcld/llmproxy/v1"
)

client.embeddings.create(
        model="text-embedding-ada-002",
        input="The food was delicious and the waiter...",
        encoding_format="float"
)
```

Client. Exactly the Part 2 set-up: Part 1 token, FIL gateway URL.

embeddings.create. One POST to /embeddings on the gateway.

Watch out. The sample never stores the result. Assign it to a variable to use it.

THE ARGUMENTS

Model, input and format

Argument	Value in the sample	What it means
model	text-embedding-ada-002	The embedding model. Must be an embedding model, not a chat model.
input	One sentence	A string, or a list of strings to embed many texts in one call.
encoding_format	"float"	Return plain decimal numbers. The other option, "base64", is compact but needs decoding.

Batch it: pass a list to input and you get one vector back per item, in the same order.

THE RESPONSE

Where the vector lives inside the result

```
response = client.embeddings.create(...)
vector = response.data[0].embedding

response
    .data[0].index → 0
    .data[0].embedding → [0.0023, -0.0091, ...]
    .model → "text-embedding-ada-002"
    .usage → prompt and total tokens

len(vector) # 1536
```

data. One entry per input text. A single string gives a list of one.

embedding. The vector itself: a Python list of 1,536 floats.

index. Matches each vector to its position in your input list.

usage. Input tokens counted. There are no output tokens.

FROM VECTORS TO MEANING

Cosine similarity: one number for how alike two texts are

HIGHER SCORE · SIMILAR MEANING

"The food was delicious"

"The meal tasted amazing"

LOWER SCORE · DIFFERENT MEANING

"The food was delicious"

"Our flight was delayed"

```
import numpy as np
score = np.dot(vector_a, vector_b) # ada-002 vectors have length 1, so dot product = cosine
```

SEMANTIC SEARCH

Four steps from a question to the best matches

1

Embed the documents

Once, up front. Store each vector next to its text.

2

Embed the question

At query time, with the same model.

3

Score every document

Cosine similarity between the question and each document.

4

Return the top matches

Show them, or pass them to the chat model as context.

Step 4 feeding Part 2's chat call is **RAG**: retrieval-augmented generation.

TRY IT

A tiny semantic search in a few lines

```
import numpy as np # client is the OpenAI client from Part 2

docs = ["The food was delicious and the waiter was friendly.",
        "Our flight was delayed by three hours.",
        "The pasta was cold but the dessert was great."]
query = "How was the meal?"

resp = client.embeddings.create(model="text-embedding-ada-002", input=docs + [query])
vectors = np.array([d.embedding for d in resp.data])
doc_vecs, query_vec = vectors[:-1], vectors[-1]
scores = doc_vecs @ query_vec # cosine similarity per document

for i in np.argsort(-scores):
    print(f"{scores[i]:.3f} {docs[i]}")
```

GOOD PRACTICE

Four rules that save you pain later

One model for everything

Documents and queries must use the same embedding model, or the scores are meaningless.

Embed once, store forever

Save document vectors with their text. Re-embed only when the text or model changes.

Chunk long text

ada-002 takes up to 8,191 tokens per input. Split long documents into passages.

Same auth rules apply

Expired token means 401; UAT token with UAT gateway. Exactly as in Parts 1 and 2.

RECAP

Text in, vector out, compare vectors to compare meaning.

Create

embeddings.create with text-embedding-ada-002 on the same client

Read

response.data[i].embedding: 1,536 floats per input

Use

Cosine similarity for search, clustering for grouping, both for RAG

FILXGPT APIS · PART 4

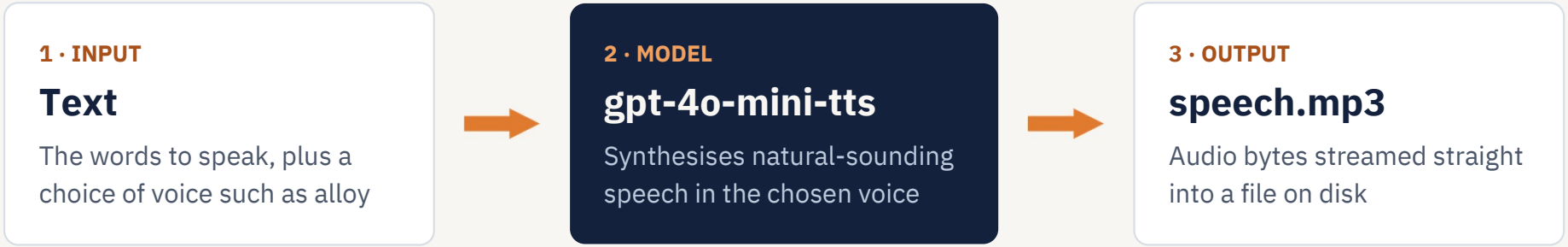
Turning text into speech

Generating an MP3 with gpt-4o-mini-tts, streaming it to a file, and the one set-up step the sample leaves out

"The quick brown fox..." → gpt-4o-mini-tts → speech.mp3

WHAT IT DOES

You send text and a voice; you get back an audio file



Unlike chat and embeddings, the answer is **binary audio**, not JSON, so we write it to a file instead of printing it.

USAGE EXAMPLES

Two reasons to give text a voice

Text-to-speech

Turn written content into natural-sounding audio for any application.

Example: narrated training modules, spoken answers from a voice assistant, audio summaries of reports.

Accessibility

Offer an audio version of text for users with visual impairments.

Example: a "listen to this page" button on an internal portal or knowledge article.

THE CODE

The sample from the docs, line by line

```
from pathlib import Path
import openai

speech_file_path = Path(__file__).parent / "speech.mp3"

with openai.audio.speech.with_streaming_response.create(
        model="gpt-4o-mini-tts",
        voice="alloy",
        input="The quick brown fox jumped over the lazy dog."
) as response:
        response.stream_to_file(speech_file_path)
```

Path. Where to save: speech.mp3 in the same folder as the script.

create(...). Model, voice and the text to speak.

stream_to_file. Writes the audio bytes to disk as they arrive.

THE CATCH

This sample never points at the FIL gateway

⚠ No access_token, no base_url. As written, the SDK falls back to its defaults: the OPENAI_API_KEY variable and OpenAI's public API.

Fix A · a client, as in Parts 2 and 3

```
client = OpenAI(api_key=access_token,
                base_url=FILXGPT_BASE_URL)
with client.audio.speech...
```

Recommended. Same pattern for every endpoint.

Fix B · configure the module

```
openai.api_key = access_token
openai.base_url = FILXGPT_BASE_URL
with openai.audio.speech...
```

Works, but global settings are easy to miss.

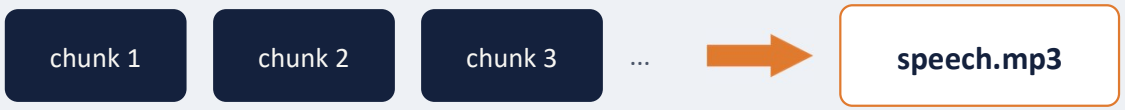
THE ARGUMENTS

Model, voice and input, plus two useful extras

Argument	Value	What it does
model	gpt-4o-mini-tts	The speech model. Required.
voice	"alloy"	One of several built-in voices, such as echo, nova or shimmer. Required.
input	The text to speak	Required. Up to 4,096 characters per request.
response_format	"mp3" (default)	Optional. Also wav, opus, aac, flac or pcm.
instructions	"Speak calmly"	Optional. Steers tone and style on gpt-4o-mini-tts.

THE STREAMING RESPONSE

Audio arrives in chunks and goes straight to disk



with_streaming_response

Hands you the response before the body is downloaded, so bytes can be written as they arrive.

with ... as response

The context manager closes the connection cleanly, even if an error occurs.

stream_to_file

Writes each chunk to the file, so the whole clip never has to sit in memory.

WHERE THE FILE GOES

Reading Path(__file__).parent / "speech.mp3"

__file__

The full path of the running script, for example /home/dev/tts_demo.py.

.parent

The folder that contains the script: /home/dev.

/ "speech.mp3"

pathlib's / joins paths: /home/dev/speech.mp3.

⚠ In a Jupyter notebook __file__ does not exist and the line raises NameError. Use Path("speech.mp3") there instead.

TRY IT

A gateway-ready version of the sample

```
import os
from pathlib import Path
from openai import OpenAI

# access_token comes from the Part 1 MSAL code
client = OpenAI(api_key=access_token, base_url=os.environ["FILXGPT_BASE_URL"])
speech_file_path = Path("speech.mp3") # works in scripts and notebooks

with client.audio.speech.with_streaming_response.create(
        model="gpt-4o-mini-tts",
        voice="alloy",
        input="Welcome to the AI foundations training.",
        instructions="Speak in a warm, upbeat tone.",
) as response:
        response.stream_to_file(speech_file_path)

print(f"Saved {speech_file_path.resolve()}")
```

WHEN IT BREAKS

Four problems you are likely to hit

Missing key or wrong host

No client configured. Pass access_token and the gateway base_url.

NameError: __file__

You are in a notebook. Use Path("speech.mp3") instead.

Text too long

Input is capped at 4,096 characters. Split long text and generate it in parts.

File will not play

The extension must match response_format, such as .wav for "wav".

RECAP

Text and a voice in, an MP3 out, through the gateway.

Connect

Add the client the sample leaves out: token plus gateway URL

Generate

audio.speech with model, voice and input

Save

Stream the bytes to a file with stream_to_file

FILXGPT APIS · PART 5

Turning speech into text

Transcribing audio with gpt-4o-transcribe, reading the result, and making recordings searchable

speech.mp3 → gpt-4o-transcribe → "The quick brown fox..."

WHAT IT DOES

You upload an audio file; you get back the words spoken in it

1 · INPUT

Audio file

A recording such as speech.mp3, sent as raw bytes

2 · MODEL

gpt-4o-transcribe

Listens to the audio and writes down what was said

3 · OUTPUT

Transcript

Plain text you can read, store, search or summarise

The mirror image of Part 4: there we sent text and got audio; here we **send audio and get text**.

USAGE EXAMPLES

Two reasons to turn audio into text

Speech-to-text

Convert spoken audio into written text for documentation or analysis.

Example: meeting minutes from a recorded call, or notes from a recorded interview.

Content indexing

Make audio searchable by transcribing it into text.

Example: find the moment in a library of training recordings where "deployment rollback" was discussed.

THE CODE

The sample from the docs, line by line

```
from openai import OpenAI

client = OpenAI(
        api_key=access_token,
        base_url="https://fil-ai-gateway-uat.k8s.npiek8sinfra1np1.npaws.ukfilcld/llmproxy/v1"
)

audio_file = open("speech.mp3", "rb")
transcript = client.audio.transcriptions.create(
        model="gpt-4o-transcribe",
        file=audio_file
)
```

Client. Good news: unlike Part 4, this sample does use the gateway client.

open(..., "rb"). Opens the audio as raw bytes for upload.

transcriptions.create. Uploads the file and returns the transcript.

OPENING THE AUDIO

Read the file as bytes, and close it when you are done

As in the sample

```
audio_file = open("speech.mp3", "rb")
transcript = client.audio...(
        file=audio_file, ...)
# file is never closed
```

Works for a quick demo, but leaves the file handle open.

Better: a with block

```
with open("speech.mp3", "rb") as audio_file:
        transcript = client.audio...(
                file=audio_file, ...)
# closed automatically here
```

Closes the file even if the API call fails. Use this in real code.

"rb" means read, binary. Audio is bytes, not text, so never open it in text mode.

THE ARGUMENTS

Two required, three worth knowing

Argument	Example	What it does
model	gpt-4o-transcribe	Required. The speech-recognition model.
file	audio_file	Required. An open audio file, such as mp3, wav, m4a or webm, up to 25 MB.
language	"en" or "hi"	Optional. Tells the model the spoken language, which can improve accuracy.
prompt	"FILxGPT, MSAL"	Optional. Hints for names, jargon and spellings.
response_format	"json" (default)	Optional. "text" returns a plain string instead.

THE RESPONSE

The words are in transcript.text

```
print(transcript.text)

The quick brown fox jumped over the lazy dog.
```

Default: an object

With the default "json" format you get a Transcription object; read its .text.

"text" format: a string

Pass response_format="text" and the result is already a plain Python string.

Round trip

Transcribe Part 4's speech.mp3 and you should get the original sentence back.

CONTENT INDEXING

From recordings to search, using Parts 2, 3 and 5 together

1

Transcribe

Audio to text with this API.

PART 5

2

Chunk

Split long transcripts into short passages.

PLAIN PYTHON

3

Embed

Turn each passage into a vector and store it.

PART 3

4

Search or summarise

Find passages by meaning, or ask the chat model about them.

PARTS 3 AND 2

Same token, same client, same gateway at every step. Only the method changes.

TRY IT

Transcribe a whole folder of recordings

```
import os
from pathlib import Path
from openai import OpenAI

client = OpenAI(api_key=access_token, base_url=os.environ["FILXGPT_BASE_URL"])

for path in sorted(Path("recordings").glob("*.mp3")):
        with open(path, "rb") as audio_file:
                transcript = client.audio.transcriptions.create(
                        model="gpt-4o-transcribe", file=audio_file, language="en",
                )
        path.with_suffix(".txt").write_text(transcript.text)
        print(path.name, "→", transcript.text[:60])
```

WHEN IT BREAKS

Four problems you are likely to hit

FileNotFoundError

"speech.mp3" is relative to the folder you run from. Check the path or use a full one.

File too large

Uploads are capped at 25 MB. Split long recordings into shorter pieces first.

Opened in text mode

open(path) without "rb" breaks on audio bytes. Always open with "rb".

Names spelled wrong

Pass a prompt listing names and jargon, and set language when you know it.

RECAP

Audio file in, transcript out, ready to search.

Open

The audio in "rb" mode, inside a with block

Transcribe

audio.transcriptions.create with gpt-4o-transcribe

Use

Read transcript.text, then store, embed or summarise it

FILXGPT APIS · PART 6

Turning words into images

Generating pictures with gpt-image-1, decoding the base64 response, and writing prompts that get what you want

"A cute baby sea otter" → gpt-image-1 → output.png

WHAT IT DOES

You describe a picture in words; the model paints it

1 · INPUT

A prompt

A text description, plus size and how many images

2 · MODEL

gpt-image-1

Generates a brand-new image that matches the description

3 · OUTPUT

output.png

Delivered as base64 text, which you decode and save

Same client and gateway as every other part. New this time: the picture arrives **encoded as text**.

USAGE EXAMPLES

Two reasons to generate images

Visual content creation

Generate unique images for marketing, social media or creative projects.

Example: a header illustration for an internal newsletter or a training slide.

Concept visualisation

Quickly visualise ideas and concepts from text descriptions.

Example: a rough mock-up of an app screen or office layout to discuss in a meeting.

THE CODE · PART 1

Ask the model for an image

```
import base64
from openai import OpenAI

client = OpenAI(
        api_key=access_token,
        base_url="https://fil-ai-gateway-uat.k8s.npiek8sinfra1np1.npaws.ukfilcld/llmproxy/v1"
)

img = client.images.generate(
        model="gpt-image-1",
        prompt="A cute baby sea otter",
        n=1,
        size="1024x1024"
)
```

base64. Standard library, needed to decode the image later.

Client. The usual gateway set-up from Part 2.

images.generate. One call; img holds the result.

THE ARGUMENTS

Four in the sample, two more worth knowing

Argument	In the sample	What it does
model	"gpt-image-1"	The image-generation model.
prompt	"A cute baby sea otter"	The description of the picture you want.
n	1	How many images to generate in one call.
size	"1024x1024"	Square. Also 1536x1024 (landscape) or 1024x1536 (portrait).
quality	not set	Optional: low, medium or high. Higher is slower and costs more.
background	not set	Optional: "transparent" for a PNG with no background.

THE CODE · PART 2

The image comes back as text, so decode it before saving

```
image_bytes = base64.b64decode(img.data[0].b64_json)
with open("output.png", "wb") as f:
        f.write(image_bytes)
```

img.data[0].b64_json
"iVBORw0KGgo..."

→

base64.b64decode
Text back into raw bytes

→

open(..., "wb")
Bytes written to output.png

Why base64? JSON can only carry text, so binary image data is encoded as letters and digits. "wb" means write, binary: the mirror of Part 5's "rb".

WRITING PROMPTS

Say what, how, and where it will be used

Subject	Style	Setting	Light, mood	Layout
a baby sea otter	watercolour	calm blue water	soft morning light	space for a title

SAMPLE PROMPT

"A cute baby sea otter"

The model guesses everything else.

MORE SPECIFIC

"A watercolour illustration of a baby sea otter floating on its back in calm blue water, soft morning light, pastel colours, empty space at the top for a title"

TRY IT

Three variations, each saved to its own file

```
import base64, os
from pathlib import Path
from openai import OpenAI

client = OpenAI(api_key=access_token, base_url=os.environ["FILXGPT_BASE_URL"])
prompt = "A watercolour baby sea otter floating in calm blue water, soft morning light"
img = client.images.generate(model="gpt-image-1", prompt=prompt, n=3, size="1536x1024")

out_dir = Path("images")
out_dir.mkdir(exist_ok=True)
for i, item in enumerate(img.data, start=1):
        path = out_dir / f"otter_{i}.png"
        path.write_bytes(base64.b64decode(item.b64_json))
        print("Saved", path)
```

WHEN IT BREAKS

Four problems you are likely to hit

Image keeps vanishing

The sample always writes output.png, so each run overwrites the last. Use unique names.

Asked for 3, got 1

The sample saves only data[0]. Loop over img.data to keep every image.

Request rejected

Prompts that break content rules return an error. Rephrase; catch the exception in code.

Slow responses

Generation takes far longer than chat. Show progress and avoid tight loops.

RESPONSIBLE USE

A generated image is easy to make and easy to misuse

Label it

Say when an image is AI-generated, especially in anything shared outside the team.

No real people or brands

Avoid images of real individuals, logos or other companies' trademarks.

Not a source of facts

Charts, diagrams and text inside images can look right and be wrong.

Follow FIL policy

Check internal rules before using generated images in client or public material.

RECAP

Prompt in, base64 out, decode and save.

Generate

images.generate with gpt-image-1, a prompt, n and size

Decode

base64.b64decode on data[i].b64_json

Save

Write the bytes in "wb" mode, with a unique file name

FILXGPT APIS · PART 7

Editing and combining images

Using images.edit with gpt-image-1 to change, enhance or merge pictures you already have

product.jpeg + person.jpg + prompt → output.png

GENERATE VERSUS EDIT

Same model, but now you start from pictures you already have

Aspect	Part 6 · Generate	Part 7 · Edit
Method	images.generate	images.edit
You send	A prompt only	One or more images plus a prompt
The prompt	Describes the whole picture	Describes the change or the combination
Optional	Nothing extra	A mask that limits where changes happen
You get	Base64 PNG	Base64 PNG, decoded the same way

USAGE EXAMPLES

Three ways to edit an image

Targeted editing

Change one region of an image, marked by a mask, using a natural-language prompt.

Example: replace the sky in a photo, leave the rest.

Variation and enhancement

Transform or improve an image while keeping its structure and context.

Example: turn a sketch into a clean illustration.

Multi-image composition

Combine several images, such as a product and a person, into one scene.

This is what the sample does.

THE CODE

The new part: images.edit

```
# client = OpenAI(api_key=access_token, base_url=...) same as Parts 2 to 6
result = client.images.edit(
        model="gpt-image-1",
        image=[
                open("moisturizer.jpeg", "rb"),
                open("srk.jpg", "rb"),
        ],
        prompt="Create a realistic ad composition using the person and the lotion
                bottle in one polished scene.",
        size="1024x1024"
)
# then base64-decode result.data[0].b64_json and write output.png, exactly as in Part 6
```

image=[...]. A list of open files: the pictures to combine.

prompt. Says what to do with them, not what they are.

Output. Same base64 handling as Part 6.

PASSING THE IMAGES

Open each file in "rb" mode, and close them afterwards

One image or many

Pass a single file to edit it, or a list of files to combine them.

Bytes, not paths

The SDK uploads open file objects, so "rb" is required, as in Part 5.

Never closed

The sample opens both files inline and never closes them.

```
# Safer: a with block closes every file, even if the call fails
with open("moisturizer.jpeg", "rb") as product, open("person.jpg", "rb") as person:
        result = client.images.edit(model="gpt-image-1", image=[product, person], prompt=prompt)
```

TARGETED EDITING

A mask says where the model may change the image



1 · Original image

2 · Mask: clear area = edit

3 · Only the sky changed

```
mask=open("mask.png", "rb"),
prompt="Replace the sky with a sunset"
```

The mask is a PNG with transparency, the same size as the image. Transparent pixels mark the area to edit.

THE ARGUMENTS

Four in the sample, two more worth knowing

Argument	In the sample	What it does
model	"gpt-image-1"	The same image model as Part 6.
image	A list of two files	The picture to edit, or several to combine. PNG, JPEG or WebP.
prompt	"Create a realistic ad..."	What to change, or how to combine the inputs.
size	"1024x1024"	Output size, with the same options as Part 6.
mask	not set	Optional PNG limiting where edits happen.
n	not set (1)	Optional: how many edited versions to return.

TRY IT

A safer version of the composition sample

```
import base64, os
from datetime import datetime
from openai import OpenAI, BadRequestError

client = OpenAI(api_key=access_token, base_url=os.environ["FILXGPT_BASE_URL"])
prompt = "Combine the person and the lotion bottle into one polished ad scene."

try:
        with open("product.jpeg", "rb") as product, open("model.jpg", "rb") as person:
                result = client.images.edit(model="gpt-image-1", image=[product, person], prompt=prompt)
except BadRequestError as err:
        raise SystemExit(f"Edit rejected: {err}")

out = f"composite_{datetime.now():%Y%m%d_%H%M%S}.png"
with open(out, "wb") as f:
        f.write(base64.b64decode(result.data[0].b64_json))
print("Composed image saved:", out)
```

WHEN IT BREAKS

Four problems you are likely to hit

Mask rejected

The mask must be a PNG with transparency and the same size as the image.

Inputs mixed up

List order does not label images. Describe each one in the prompt.

Details drift

Faces, product labels and logos can come back subtly changed. Always review the output.

Output overwritten

The sample always writes output.png. Use unique names, as in the script.

RESPONSIBLE USE

Editing real photos raises the stakes

Consent for people

Putting a real person in an ad, as the sample does, needs their permission and the image rights.

Own the inputs

Only upload photos and product images you are allowed to use and share.

Disclose edits

Label AI-edited images, especially in marketing and anything public.

Don't mislead

An edited ad must not imply an endorsement or a product result that isn't real.

RECAP

Images and a prompt in, one new image out.

Send

images.edit with one or more files opened in "rb", closed afterwards

Steer

A prompt that names each input, plus an optional mask

Save and check

Decode as in Part 6, review details, and confirm rights

FILXGPT APIS · PART 9

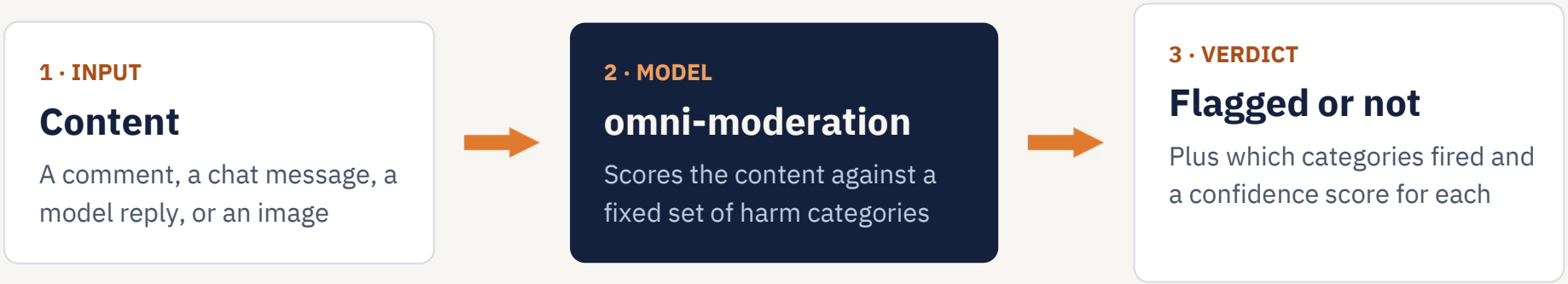
Checking content for harm

Using the moderation API to screen text and images before they reach your users or your models

text or image → omni-moderation → flagged? which categories? how sure?

WHAT IT DOES

A classifier, not a generator: it judges content and reports back



Moderation **informs** a decision; your application decides what to do: block, hold for review, or allow.

USAGE EXAMPLES

Four places moderation earns its keep

Content safety filtering

Screen user content before you display or store it.

Policy enforcement

Flag content that breaks guidelines across many harm categories.

Multi-modal review

Check text and images, with a score for each category.

Real-time moderation

Instant decisions inside chats, comments and submission forms.

THE CODE

The sample from the docs, line by line

```
# client = OpenAI(api_key=access_token, base_url=...) same as Parts 2 to 8

moderation = client.moderations.create(
        model="openai/omni-moderation-latest",
        input="Sample text to check for harmful content."
)

print(moderation.results[0])
print(f"Flagged: {moderation.results[0].flagged}")
print(f"Categories: {moderation.results[0].categories}")
```

model. Note the "openai/" prefix: the gateway's name for the model.

input. The text to check. A list checks several at once.

results[0]. One result per input; [0] is the first.

READING THE RESULT

One verdict, a yes/no per category, and a score per category

```
moderation.results[0]
    .flagged → False
    .categories → hate=False, violence=False, ...
    .category_scores → hate=0.0001, violence=0.0003, ...
    .category_applied_input_types → text / image
```

flagged. True if any category crossed the model's default line.

categories. True or False for each harm category.

category_scores. Confidence from 0 to 1. Use these for your own thresholds.

Printing categories shows a long object. Use **.model_dump()** to get a plain Python dict you can loop over.

THE CATEGORIES

Six families of harm, some with stricter sub-categories

Family	Category names	Covers
Harassment	harassment, harassment/threatening	Abuse aimed at a person, including threats
Hate	hate, hate/threatening	Attacks on people for who they are
Illicit	illicit, illicit/violent	Advice on wrongdoing, including with weapons
Self-harm	self-harm, self-harm/intent, self-harm/instructions	Content about hurting oneself
Sexual	sexual, sexual/minors	Sexual content, with a separate category for minors
Violence	violence, violence/graphic	Depictions of violence, and graphic detail

MULTI-MODAL REVIEW

Check an image and its caption in one call

```
moderation = client.moderations.create(
        model="openai/omni-moderation-latest",
        input=[
                {"type": "text", "text": caption},
                {"type": "image_url", "image_url": {"url": image_url}},
        ],
)
```

The image

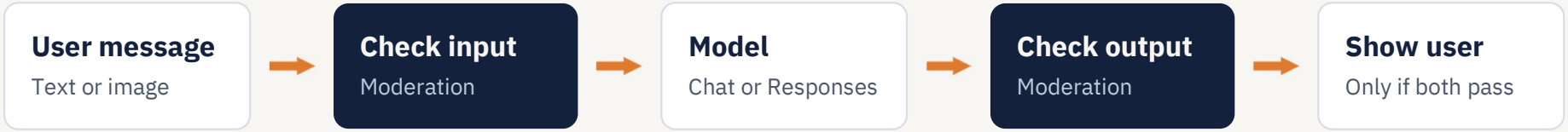
A web URL, or a base64 data URL as in Parts 6 and 7.

Not every category

Some categories assess text only; check applied_input_types.

WHERE IT FITS

Check on the way in, and again on the way out



Input check

Stops harmful requests before they cost a model call.

Output check

Catches a model reply that slipped past its own safeguards.

YOUR POLICY, YOUR THRESHOLDS

Turn scores into three outcomes, not just yes or no



Score 0

Above your review line, e.g. [0.__]

Above your block line, or flagged

Tune on real data

Pick cut-offs by testing on examples from your own app.

Stricter where it matters

Use lower lines for the most serious sub-categories.

People decide edge cases

The middle band goes to a reviewer, not straight to a block.

TRY IT

A reusable moderation check for comments

```
REVIEW_AT, BLOCK_AT = 0.2, 0.7 # example values only; tune on your own data

def check(text: str) -> tuple[str, list[str]]:
        result = client.moderations.create(
                model="openai/omni-moderation-latest", input=text
        ).results[0]
        scores = {k: v for k, v in result.category_scores.model_dump().items() if v is not None}
        top = sorted(scores, key=scores.get, reverse=True)[:3]
        if result.flagged or max(scores.values()) >= BLOCK_AT: return "block", top
        if max(scores.values()) >= REVIEW_AT: return "review", top
        return "allow", []

decision, reasons = check("Great session today, thanks everyone!")
print(decision, reasons) # expected: allow []
```

WATCH OUT FOR

Moderation is a strong filter, not a guarantee

False alarms and misses

Sarcasm, quotes and domain jargon can fool it either way. Keep a human review path.

Not your company policy

It knows general harm, not FIL rules such as confidential data. Add your own checks.

Wrong model name

Use the gateway's name, "openai/omni-moderation-latest", exactly as documented.

Careless logging

Flagged content may be sensitive. Log decisions and scores, not the raw text.

RECAP

The API scores the content; your code makes the call.

Check

moderations.create with text, or text plus an image

Read

flagged, categories and category_scores on results[0]

Decide

Allow, review or block, on the way in and the way out

FILXGPT APIS · PART 10

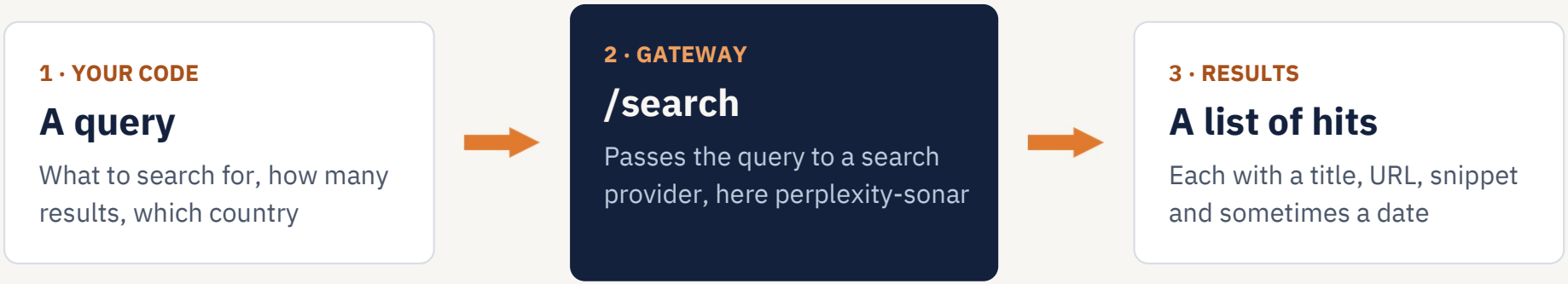
Searching the live web

Calling the gateway's /search endpoint, reading the results, and grounding model answers in up-to-date sources

query → /search (perplexity-sonar) → titles, URLs, snippets

WHAT IT DOES

The gateway runs a web search for you and returns the hits



This is a **FIL gateway endpoint**, not a standard OpenAI API, so the SDK has no ready-made method for it.

USAGE EXAMPLES

Five uses, each driven by a different setting

Use case	What it means	Main lever
Real-time information	Current news, events and data for AI answers	query
Research and fact-checking	Verify facts across sources, with citations	max_results
Domain-specific search	Only trusted sites, such as academic or news	A domain filter
Localized discovery	Region-specific news and content	country
Content aggregation	Digests and overviews from many sources	Snippets plus a model

THE CODE · PART 1

The request, as written in the docs

```
# client = OpenAI(api_key=access_token, base_url=...) same as Parts 2 to 9
response = client.post(
        '/search',
        json={
                "search_tool_name": "perplexity-sonar",
                "query": "latest AI news 2026",
                "max_results": 10,
                "country": "US"
        }
)
```

client.post. A raw POST, since no named method exists.

'/search'. Added to base_url: .../llmproxy/v1/search.

json={...}. The settings. Read the next slide first.

THE CATCH

As written, client.post will likely fail

⚠ In the OpenAI Python SDK, post() expects **body=** and a required **cast_to=**. Written as in the docs, expect a TypeError.

Fix A · Keep the SDK

```
data = client.post(
        "/search",
        body=payload,
        cast_to=object,
)
```

Uses an internal-style method; check the result type in your SDK version.

Fix B · Plain HTTP (recommended)

```
resp = requests.post(
        f"{BASE_URL}/search", json=payload,
        headers={"Authorization": f"Bearer {token}"},
)
data = resp.json()
```

Same token as a Bearer header, as in Part 1. Clear and predictable.

THE SETTINGS

Four fields shape the search

Field	In the sample	What it does
search_tool_name	"perplexity-sonar"	Which search provider the gateway uses.
query	"latest AI news 2026"	What to search for. Be as specific as you would in a search engine.
max_results	10	How many results to return. Fewer is faster and cheaper to process.
country	"US"	Two-letter country code to favour local results, such as "IN".

Filtering to trusted domains is listed as a use case, but its field name is not in the sample. **Check the FIL docs.**

THE CODE · PART 2

Loop over the results and print each hit

```
for result in response['results']:
        print(f"Title: {result['title']}")
        print(f"URL: {result['url']}")
        print(f"Snippet: {result['snippet']}")
        print(f"Date: {result.get('date', 'N/A')}")
        print("---")
```

results. A list, one dict per web page found.

title, url, snippet. Always present. The snippet is a short extract, not the full page.

.get('date'). Not every page has a date, so .get avoids a KeyError.

GROUNDING

Search, then let a model answer from the sources

1 · Search (this part) → 2 · Number the snippets → 3 · Answer with citations (Part 8)

```
sources = "\n".join(f"[{i}] {r['title']} ({r['url']}): {r['snippet']}" for i, r in enumerate(results, 1))

answer = client.responses.create(
        model="gpt-4.1",
        instructions="Answer only from the sources. Cite them as [1], [2]. Say if they don't cover it.",
        input=f"Question: {question}\n\nSources:\n{sources}",
)
print(answer.output_text)
```

TRY IT

A reusable web_search helper

```
import os, requests
BASE_URL = os.environ["FILXGPT_BASE_URL"]

def web_search(query, max_results=5, country="US"):
        resp = requests.post(
                f"{BASE_URL}/search",
                headers={"Authorization": f"Bearer {access_token}"},
                json={"search_tool_name": "perplexity-sonar", "query": query,
                        "max_results": max_results, "country": country},
                timeout=30,
        )
        resp.raise_for_status()
        return resp.json().get("results", [])

results = web_search("RBI digital lending guidelines 2026", country="IN")
```

SAFETY

Web search opens a door in both directions

Queries leave FIL

They go to an outside search provider. Never put client names or confidential data in a query.

Web text is untrusted

A page can hide instructions aimed at your model. Treat snippets as data, never as commands.

Verify before you rely

Results can be wrong, biased or out of date. Prefer trusted domains and check key facts.

Keep the citations

Show URLs with every answer so readers can trace each claim.

WHEN IT BREAKS

Four problems you are likely to hit

TypeError on client.post

The SDK's post() has no json= argument. Use Fix A or Fix B.

KeyError on a field

Optional fields like date can be missing. Use .get() with a default.

Nothing relevant comes back

Make the query specific, add a year, or change the country.

App hangs

Web search can be slow. Always set a timeout and handle the error.

RECAP

Query in, fresh sources out, a model to answer from them.

Search

POST to the gateway's /search with query, max_results and country

Read

title, url and snippet for each result; date if present

Ground

Feed numbered snippets to a model and keep the citations

FILXGPT APIS · PART 11

Vector stores: a managed home for your documents

Create, inspect, update, delete, list and search the knowledge bases behind RAG

create · retrieve · update · delete · list · search

WHAT IS A VECTOR STORE?

Part 3's semantic search, run for you as a service



The dark steps are **done for you**. You choose the settings, add documents, and ask questions.

THE SIX OPERATIONS

One store, six things you can do with it

Operation	Call	Use it for
Create	vector_stores.create	Provision a knowledge base with chunking and expiry
Retrieve	vector_stores.retrieve	Check status, file counts and storage used
Update	vector_stores.update	Rename, change expiry, add metadata
Delete	vector_stores.delete	Clean up, refresh or meet retention rules
List	vector_stores.list	Discover and audit all stores
Search	POST .../vector_stores/{id}/search	Find the best passages for a query

1 · CREATE

Provision a knowledge base

```
vector_store = client.vector_stores.create(
        name="Policy Knowledge Base",
        chunking_strategy={
                "type": "static",
                "static": {"max_chunk_size_tokens": 800,
                                        "chunk_overlap_tokens": 400},
        },
        expires_after={"anchor": "last_active_at", "days": 30},
)
print(vector_store.id) # e.g. vs_abc123
print(vector_store.status)
```

name. A label for people. The id is what code uses.

chunking_strategy. How documents are split. Next slide.

expires_after. Auto-delete after 30 idle days.

id. Save it; every other call needs it.

CUSTOM CHUNKING CONTROL

800-token chunks that overlap by 400



Chunk size

Bigger keeps context together; smaller makes matches more precise.

Overlap

A sentence cut at a boundary still appears whole in the next chunk.

The trade-off

50% overlap means about twice the chunks to store and search.

LIFECYCLE AND COST MANAGEMENT

expires_after: idle stores clean themselves up

Last used
last_active_at is updated on use

→

30 idle days
Any use restarts the clock

→

Expired
Status becomes expired; no longer usable

Great for experiments
Training and test stores vanish on their own instead of piling up.

Risky for production
A rarely used store can expire. Use a longer period, or none.

2 · RETRIEVE

Is the store ready, and how big is it?

```
vector_store = client.vector_stores.retrieve("vs_abc123")
print(vector_store.id, vector_store.name, vector_store.status)
print(vector_store.file_counts, vector_store.usage_bytes)
```

status

in_progress while files are processed; completed when ready to search; expired.

file_counts

How many files are in_progress, completed, failed, cancelled, and the total.

usage_bytes

Storage used, for capacity planning and cost.

3 · UPDATE

Change settings without recreating the store

```
updated_vs = client.vector_stores.update(
        "vs_abc123",
        name="Updated Knowledge Base",
        expires_after={"anchor": "last_active_at", "days": 60},
        # metadata={"owner": "policy-team", "env": "uat"}, optional
)
print(updated_vs.id, updated_vs.name, updated_vs.status)
```

Rename

Same id and documents, new label.

Extend expiry

30 days becomes 60 as needs change.

Tag with metadata

Owner, team, environment for governance.

4 · DELETE

Remove a store permanently

```
deletion_result = client.vector_stores.delete("vs_abc123")
print(f"ID: {deletion_result.id}")
print(f"Deleted: {deletion_result.deleted}") # True
```

Use it for

Cleanup, retiring test stores, and refreshing a store with new documents.

No undo

There is no recycle bin. Double-check the id first, ideally by name.

Files may remain

Deleting the store does not delete uploaded files. For compliance, delete those too.

5 · LIST

See every store, newest first

```
vector_stores = client.vector_stores.list(
        limit=20, order="desc"
)
for vs in vector_stores.data:
        print(vs.id, vs.name, vs.status, vs.created_at)
```

```
# all pages, not just the first 20:
for vs in client.vector_stores.list(limit=20):
        print(vs.id, vs.name)
```

limit. Stores per page; 20 here.

order="desc". Newest created first.

.data. Only the first page. With more stores, the rest are missed.

Loop the list itself. The SDK fetches the next pages for you.

6 · SEARCH

Ask a question, get back the best passages

```
url = f"{host}/llmproxy/v1/vector_stores/{vector_store_id}/search"
headers = {"Authorization": f"Bearer {access_token}", "Content-Type": "application/json"}
data = {
        "query": "What are the document retention rules?",
        "max_num_results": 5, "rewrite_query": True,
        "ranking_options": {"ranker": "auto", "score_threshold": 0.2},
}
response = requests.post(url, headers=headers, json=data)
response.raise_for_status()
```

Plain HTTP. Same pattern as Part 10's Fix B.

Token. Use the Part 1 token; never paste a real one into code.

Results. response.json()["data"]: score, filename and text.

RETRIEVAL TUNING

Four knobs control what comes back

Setting	Sample	Effect
max_num_results	5	How many passages to return. More context, but more noise.
rewrite_query	True	Lets the service rephrase your question into a better search query.
ranker	"auto"	Which ranking method orders results. Auto picks the default.
score_threshold	0.2	Drops passages scoring below this, from 0 to 1. Higher is stricter.

Tune with real questions: too few results misses answers; a high threshold can return **nothing at all**.

PUTTING IT TOGETHER

Add a document, then let the model use file_search

```
# 1. add a document to the store created earlier (vs)
with open("retention_policy.pdf", "rb") as f:
        client.vector_stores.files.upload_and_poll(vector_store_id=vs.id, file=f)

# 2. ask a question; the model searches the store itself (Part 8)
response = client.responses.create(
        model="gpt-4.1", input="How long must we keep client emails?",
        tools=[{"type": "file_search", "vector_store_ids": [vs.id]}],
)
print(response.output_text)
```

The missing step. None of the six samples adds documents; this does.

Full RAG. Store, retrieve and answer in one Responses call.

PITFALLS

Four mistakes to catch in code review

Searching too early

Check status is "completed" and file_counts.failed is 0 before you search.

Chunking set too late

The create-time strategy covers files in that call. Pass it again when adding files later.

Trusting .data

It holds one page only. Loop over the list itself for audits and cleanup.

Leftovers and secrets

Deleting a store leaves its files. Never hard-code a real token in a script.

RECAP

Create it, check it, keep it tidy, search it

Provision

create with a name, a chunking strategy and an expiry policy.

Manage

retrieve for status, update names, list every page, delete with care.

Use

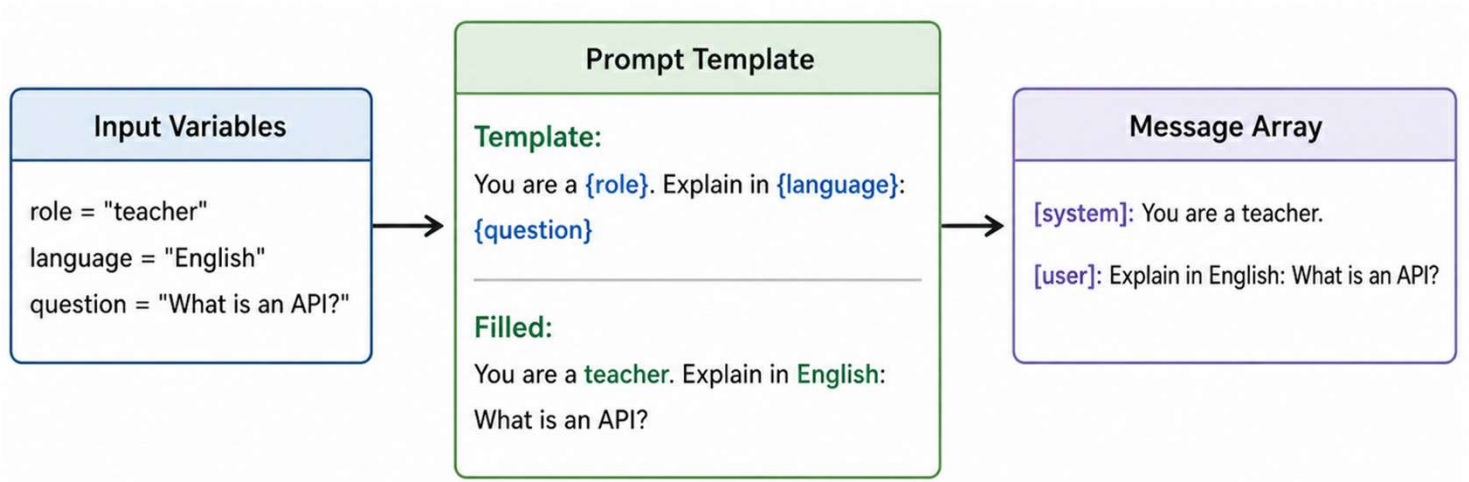
search for passages yourself, or hand the store to file_search.

PART 03

Building LLM applications

The model is fixed; what you control is the input. Templates, structure and conversation history are how you engineer reliable behaviour.

Prompt templates turn ad-hoc strings into versioned, testable code



Separate logic from data

The template is fixed; variables change per request. One template serves thousands of calls consistently.

Version and review prompts

Keep templates in source control, peer-review them and A/B test them like any other code change.

Keep untrusted input contained

Insert user text only into its slot in the user message, never into system instructions, to limit prompt injection.

Anatomy of an enterprise-grade prompt: six building blocks

1 **Role & goal** `system`
Who the model is and what success looks like.
"You are an assistant that summarises client service tickets for operations analysts."

2 **Context** `system / user`
Facts the model needs: retrieved documents, data, user profile.
"Ticket history: {ticket_thread}"

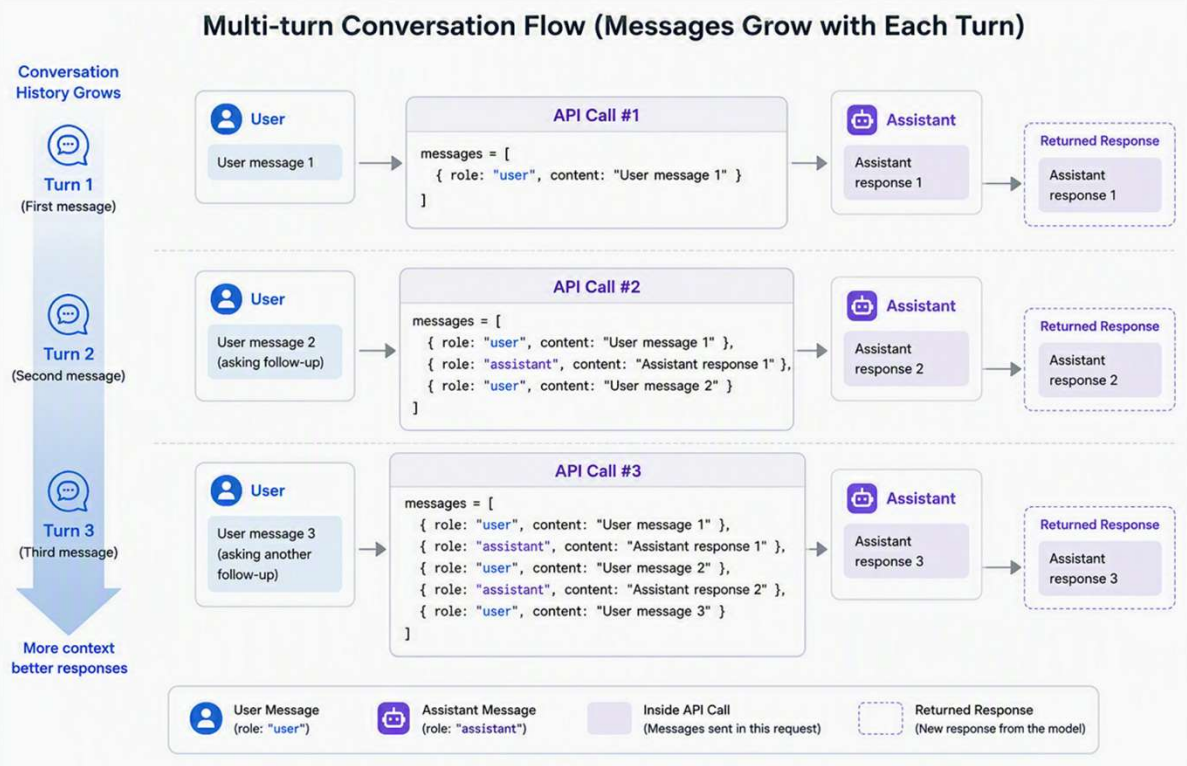
3 **Task** `user`
A single, explicit instruction.
"Summarise the issue and the next action."

4 **Constraints** `system`
Rules, tone, scope and what to refuse.
"Use only the ticket text. If unsure, say so. Never include account numbers."

5 **Output format** `system`
Exact structure, ideally a JSON schema.
"Return JSON: {summary, next_action, confidence}."

6 **Examples** `system / few-shot`
One to three worked input→output pairs.
"Example ticket … → Example summary …"

The model is stateless: your app resends the conversation every turn



No server-side memory

Each API call is independent. The illusion of a conversation comes from your app sending the full messages array again.

Context grows every turn

Turn 3 resends turns 1 and 2. Tokens, cost and latency climb, and eventually the context window is full.

Manage it deliberately

Keep the last N turns, summarise older turns, or retrieve only relevant history. Always keep the system message.

Why this matters for agents
Short-term memory is just this array. Long-term memory needs a store, covered in Part 4.

FILXGPT APIS · PART 12

RAG: answers grounded in your own documents

How retrieval-augmented generation works at FIL, how it improved in four stages, and how to measure it

ingest · retrieve · re-rank · augment · generate · evaluate

WHY RAG?

An LLM knows its training data, not your policies

The problem

Ask about an internal policy and the model has never seen it. It may still answer, and make the answer up.

The idea

Before asking, fetch the relevant passages and put them in the prompt. The model answers from evidence.

The result

Answers that are factual, domain-specific and current, with no retraining.

RAG = Retrieval-Augmented Generation: retrieve chunks, augment the prompt, generate the answer.

THE BIG PICTURE

From question to grounded answer in six steps



Retrieval: query, store, chunks. **Generation:** augment, LLM, grounded answer.

HOW RAG WORKS

Two phases: find the right chunks, then answer from them

RETRIEVAL PHASE

User query
A natural-language question

→

Vector store
Chroma (HNSW), BM25 + semantic search

→

Chunks
Re-ranked with RRF, top 5 kept

GENERATION PHASE

Prompt augmentation
Query + chunks merged into a template

→

LLM
GPT-4 / GPT-4 Turbo, 128K context

→

Grounded answer
Accurate, cited, checkable

AUGMENTATION IN CODE

The "augmented" prompt is just a template

```
TEMPLATE = """Answer using only the context below.
If the answer is not there, say you don't know.
Context: {context}
Question: {question}"""

chunks = retrieve(question, top_k=5) # retrieval phase
prompt = TEMPLATE.format(context="\n\n".join(chunks), question=question)
response = client.chat.completions.create(
        model="gpt-4-turbo",
        messages=[{"role": "user", "content": prompt}])
print(response.choices[0].message.content)
```

"Only the context". Keeps the model on the evidence.

"Say you don't know". A safe exit instead of a guess.

DATA INGESTION

Before any question: load, split, embed, store

Step	At FIL	Why it matters
Load	PDF, Word, CSV, web pages, Confluence	Many sources feed one store
Split	RecursiveCharacterTextSplitter, 500 tokens	Breaks at paragraphs first, then sentences
Embed	text-embedding-ada-002	One vector per chunk, as in Part 3
Store	Chroma with an HNSW index	Fast approximate nearest-neighbour search
Keep text	Chunk text saved as metadata	Makes BM25 keyword search possible too

FOUR STAGES

Each stage fixes the weakness of the one before

STAGE 1

Basic RAG

Embed the question, take the 4 most similar chunks.

STAGE 2

+ Multi-query

The LLM rewrites the question 3 ways. Fixes badly worded questions.

STAGE 3

+ Re-ranking

Merge semantic and keyword ranks with RRF. Fixes too many chunks.

STAGE 4

+ Intent

Route "summarise" or "translate" differently. Fixes whole-document tasks.

FIL's pipeline today is **all four stages stacked**. The next slides take them one at a time.

STAGE 1 · BASIC RAG

One question, one search, the top 4 chunks

Setting	Value
Chunking	RecursiveCharacterTextSplitter, 500 tokens
Embeddings	text-embedding-ada-002
Store	Chroma, HNSW nearest-neighbour index
Threshold	Similarity at least 0.7
Top K	4 chunks sent to the LLM

Where it breaks

People don't phrase questions the way documents are written. One phrasing can miss the passage that holds the answer.

STAGE 2 · MULTI-QUERY RETRIEVER (MQR)

Ask the same question three ways, merge the results



Gain. Much better recall for poorly worded questions.

Cost. An extra LLM call, and many more chunks to send.

STAGE 3 · RE-RANKING WITH RRF

Rank two ways, reward chunks that do well in both

RECIPROCAL RANK FUSION

score = Σ weight / (k + rank)

Semantic rank: cosine similarity, weight 0.5

BM25 rank: keyword match, weight 0.5

Keep: the top 5 by fused score

CHUNKS SENT TO THE LLM

14,644

with MQR alone

3,938

after RRF, about 73% fewer

RRF WORKED EXAMPLE

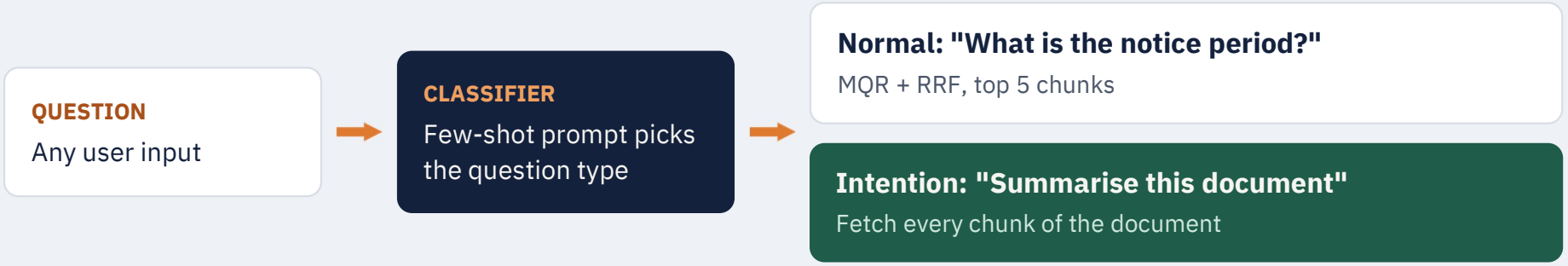
A keyword match can lift a chunk up the list

Chunk	Semantic rank	BM25 rank	Fused score	Final
A	1	3	0.5/61 + 0.5/63 = 0.0161	1st
B	2	not found	0.5/62 = 0.0081	3rd
C	4	1	0.5/64 + 0.5/61 = 0.0160	2nd

C was only 4th by meaning, but it is the **best keyword match**, so it overtakes B. (Here k = 60 and both weights are 0.5.)

STAGE 4 · INTENT CLASSIFICATION

Some questions aren't searches at all



"Summarise" has no keywords to search for. Similarity search would return **a few random chunks**.

RESPONSE GENERATION

One pipeline, many document tasks

What teams build on it

- Chat with documents: Q&A on indexed content
- Summarisation and translation
- Entity extraction (FILExtractGPT)
- Guided document exploration

Model	Context window
GPT-3.5	4K / 16K tokens
GPT-4	8K / 32K tokens
GPT-4 Turbo	128K tokens

The template, the chunks and the answer must **all fit in the context window**.

CHOOSING A TECHNIQUE

Prompting, RAG or fine-tuning?

	Prompt engineering	RAG	Fine-tuning
How	Instructions and examples	Passages added to prompt	Retrain on a small corpus
New knowledge?	No	Yes, from your documents	Little: data too small
New documents	Nothing to update	Index them, usable at once	Retrain every time
Best at	Fast iteration, few-shot	Q&A, summaries	Tone, style, format
Main limit	Can't fill knowledge gaps	Only as good as retrieval	Cost; still hallucinates

Not either-or: **combine them**, e.g. few-shot prompts inside a RAG template.

AUTOMATED EVALUATION

Score retrieval and generation separately

The same question can give different wording each run, so exact-match tests fail. Use metric frameworks: **RAGAS** or **DeepEval**.

RETRIEVAL: DID WE FIND THE RIGHT CHUNKS?

Context precision
How much of what we retrieved is relevant

Context recall
How much of what was needed we retrieved

GENERATION: IS THE ANSWER GOOD?

Faithfulness
Are its claims supported by the chunks?

Answer correctness
F1 of key points vs the ground-truth answer

Answer relevancy
Does it actually address the question?

WHEN TO REACH FOR RAG

Four rules of thumb, and which API to use

- Answers must use current internal knowledge
- Prefer RAG over fine-tuning for knowledge
- Put few-shot examples in the RAG template
- Evaluate retrieval and answers separately

v1 APIs

Multi-provider flexibility: build the pipeline yourself

v2 APIs

OpenAI-native vector stores and files, as in Part 11

PITFALLS

Six ways a RAG pipeline goes wrong

Garbage in, garbage out

Bad chunks give bad answers, however good the model.

Split facts

A fact cut across two chunks may never be retrieved.

No intent check

"Summarise this" needs every chunk, not the top 5.

MQR without re-ranking

14,644 chunks instead of 3,938: cost explodes.

Cosine alone

Misses exact terms. Add BM25 and fuse with RRF.

Exact-match testing

Output varies run to run. Score it with RAGAS metrics.

RECAP

Retrieve well, and the model answers well

Ingest

Split into chunks, embed them, keep the text for keyword search.

Retrieve

MQR for recall, RRF for precision, intent to pick the route.

Generate and measure

Answer from a grounded template; score faithfulness and correctness.

FILXGPT APIS · PART 13

AI agents: models that decide what to do next

What an agent is, how the loop works, building one by hand, and choosing a framework

model · instructions · tools · memory · loop

WHAT IS AN AI AGENT?

A system that completes tasks on its own, or nearly

> "AI agent refers to a system that is able to autonomously (or semi-autonomously) complete specific or open-ended tasks."

Reasons and plans
Breaks a goal into steps and decides what to do next

Uses tools
Calls APIs, databases and services to act

Reads its environment
Looks at tool results and adapts its plan

Needs little supervision
Runs many steps without a human prompting each one

WHERE AGENTS FIT

The real question: who decides the next step?

YOU DECIDE

Single LLM call

One prompt, one answer.

e.g. summarise this email (Part 2)

YOUR CODE DECIDES

Workflow

Fixed steps; the LLM fills each one in.

e.g. the RAG pipeline (Part 12)

THE MODEL DECIDES

Agent

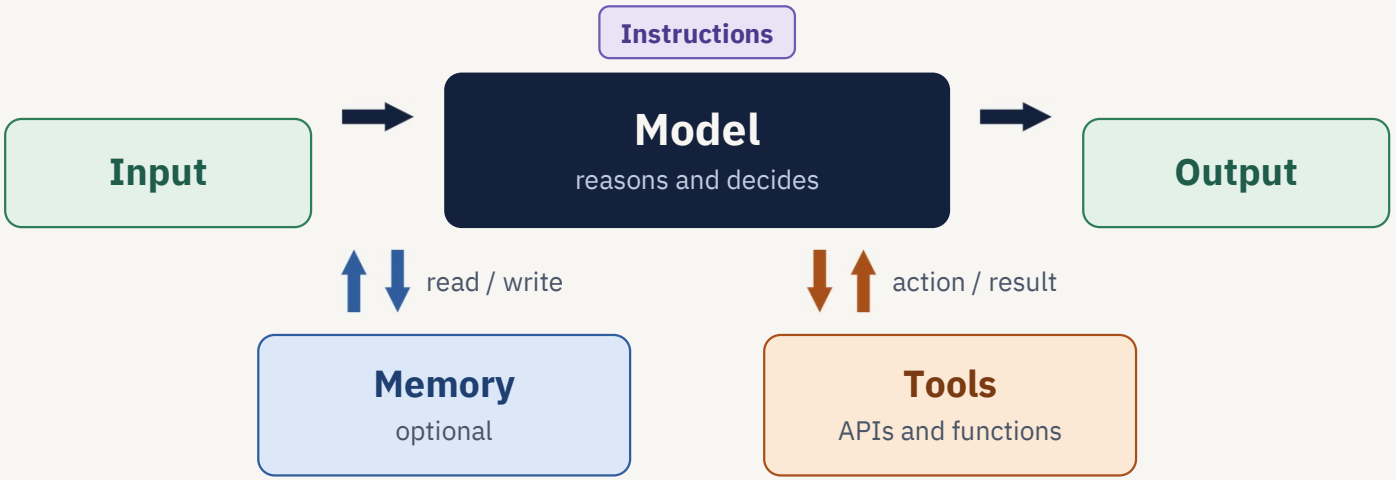
Chooses steps and tools in a loop.

e.g. investigate a failed job, then draft a ticket

Use the **simplest option that works**. More autonomy brings more power, cost and risk.

ANATOMY OF AN AGENT

A model at the centre, with memory and tools



FOUR CORE COMPONENTS

Our running example: a fund-data assistant

Component	What it is	In our assistant
Model	The LLM that reasons and decides	gpt-4.1 through the FILxGPT gateway
Instructions	Guidelines and guardrails for behaviour	"Use tools for any figure. Never guess a price."
Tools	Functions or APIs it can call to act	get_fund_price(fund_code)
Memory (optional)	What it remembers across turns	The conversation so far, kept in a list

Same model, different instructions and tools: **a completely different agent**.

THE AGENT LOOP

Think, act, observe, repeat



ONE REAL TRACE

user What is the price of fund ABC123?

think I need a live figure, so call the price tool

act get_fund_price(fund_code="ABC123")

observe {"price": 12.47, "currency": "GBP"}

answer Fund ABC123 is priced at 12.47 GBP.

BUILD IT BY HAND · STEP 1

A tool is a function plus a description

```
def get_fund_price(fund_code: str) -> dict:
    return {"fund_code": fund_code, "price": 12.47, "currency": "GBP"} # demo data

tools = [{"type": "function", "function": {
    "name": "get_fund_price",
    "description": "Latest unit price for a fund, by its fund code",
    "parameters": {"type": "object",
                   "properties": {"fund_code": {"type": "string"}},
                   "required": ["fund_code"]}}}]
```

name + description. What the model reads to choose a tool.

parameters. A JSON Schema for the arguments.

Who runs it? Your code. The model only asks.

BUILD IT BY HAND · STEP 2

The loop: about a dozen lines of Python

```
messages = [{"role": "system", "content": "Use tools for any figure. Never guess a price."},
                        {"role": "user", "content": "What is the price of fund ABC123?"}]
for step in range(5): # hard stop: never loop forever
        resp = client.chat.completions.create(model="gpt-4.1", messages=messages, tools=tools)
        msg = resp.choices[0].message
        if not msg.tool_calls: # THINK: no tool needed, so this is the answer
                print(msg.content); break
        messages.append(msg) # keep the model's tool request in memory
        for call in msg.tool_calls: # ACT
                args = json.loads(call.function.arguments)
                result = get_fund_price(**args)
                messages.append({"role": "tool", "tool_call_id": call.id, # OBSERVE
                                                "content": json.dumps(result)})
```

messages = memory. Resent on every call.

range(5) = guardrail. No endless loops.

DEVELOPMENT APPROACHES

Two ways to build an agent

FOR DEVELOPERS · SUPPORTED AT FIL

Pro-code

- Written in code with an SDK or framework
- Full control of tools, logic and data
- Version control, tests, code review

FOR NON-TECHNICAL USERS

Low-code / no-code

- Built in a visual designer
- Fast to start, little coding
- Limited when logic gets complex

FIL's AI platform supports **pro-code agents**, so the rest of this deck is about code.

FRAMEWORKS AND SDKS

Four ways to skip writing the loop yourself

LangGraph

pip install langgraph

A graph of steps with shared state. Best for complex flows that need detailed control.

Google ADK

pip install google-adk

Enterprise multi-agent orchestration with MCP. Best for large-scale systems.

OpenAI Agents SDK

pip install openai-agents

Few concepts, human-in-the-loop, tracing. Best for quick experiments.

AutoGen

pip install autogen-agentchat

Multi-agent automation. Now in maintenance mode; see notes.

Install names updated: the doc's **pip install openai** and **pyautogen** install the base SDK and the old AutoGen package.

SIDE BY SIDE

Same job, different levels of abstraction

	LangGraph	Google ADK	OpenAI Agents SDK	AutoGen
Think of it as	A graph of steps	A team of agents	Agent + tools + handoffs	Agents in conversation
Code you write	Most, lower-level	Little	Least	Moderate
Multi-agent	Yes, you wire it	Core feature	Via handoffs	Core feature
Eval and tracing	Via LangSmith	Built in	Built in	Check docs
Best for	Detailed control	Enterprise scale	Quick experiments	Existing code

SAME AGENT · OPENAI AGENTS SDK

The framework runs the loop for you

```
from openai import AsyncOpenAI
from agents import Agent, Runner, function_tool, set_default_openai_client
from agents import set_default_openai_api, set_tracing_disabled

set_default_openai_client(AsyncOpenAI(api_key=access_token, base_url=GATEWAY_URL))
set_default_openai_api("chat_completions") # the API we've used since Part 2
set_tracing_disabled(True) # tracing would send run data to OpenAI

@function_tool
def get_fund_price(fund_code: str) -> dict:
    """Latest unit price for a fund, by its fund code."""
    return {"fund_code": fund_code, "price": 12.47, "currency": "GBP"}

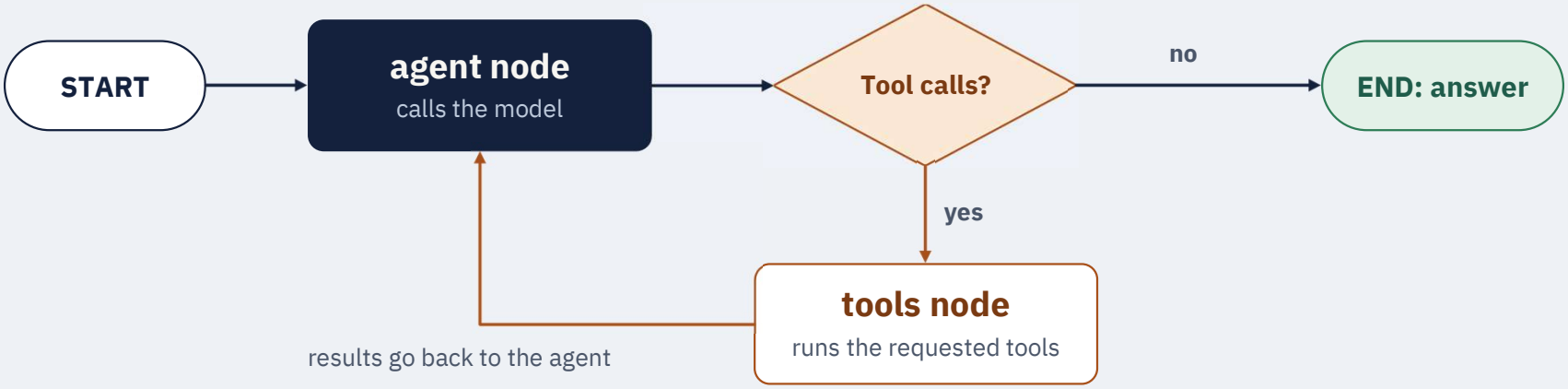
agent = Agent(name="Fund assistant", model="gpt-4.1", tools=[get_fund_price],
              instructions="Use tools for any figure. Never guess a price.")
print(Runner.run_sync(agent, "What is the price of fund ABC123?").final_output)
```

@function_tool. Builds the JSON schema from type hints and docstring.

Runner. Is the loop from step 2.

SAME AGENT · LANGGRAPH'S VIEW

The loop drawn as a graph you control



State (the message list) flows along every edge. You draw the graph; LangGraph runs it.

SELECTION CRITERIA

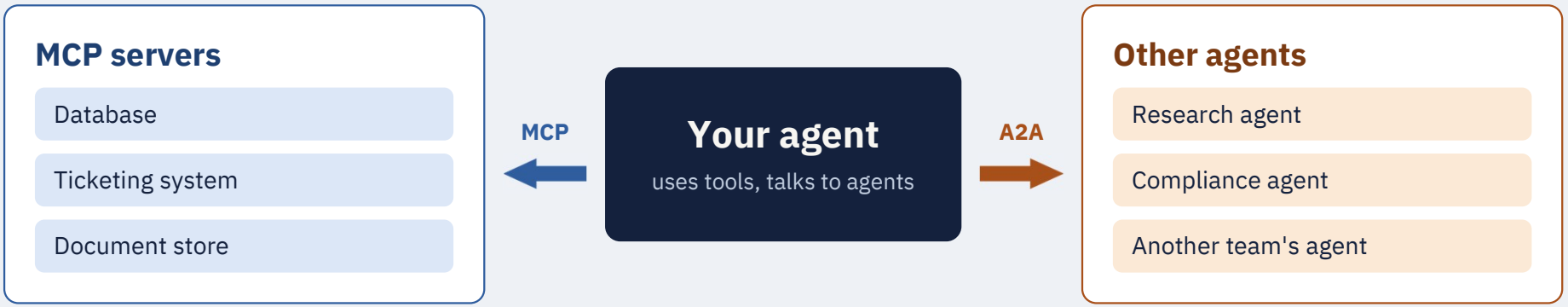
Pick by the problem, not by popularity

If you need...	Start with	Because
A quick experiment or prototype	OpenAI Agents SDK	Least code, fast iteration
Branching flows, approval steps, full control	LangGraph	You design every step
Many cooperating agents at scale	Google ADK	Built for orchestration
Built-in evaluation and tracing	ADK or Agents SDK	Both ship with them
To maintain existing AutoGen code	AutoGen	Plan a migration path

Before any of these: **check what the FILxGPT platform supports** and approves.

AGENT PROTOCOLS

MCP connects tools; A2A connects agents



Both are open standards: **build an integration once** and any compatible agent can use it.

AGENT EVALUATION

Judge the path, not just the final answer

Task completion
Did it achieve the goal?

Tool use
Right tool, right arguments, no wasted calls?

Reasoning
Were the steps sensible and in order?

Safety
Did it stay within its rules?

Golden tasks
20 to 50 real tasks with known good outcomes

Trace review
Read the logged steps, like our trace slide

Automated checks
Assert expected tool calls; LLM judges for text

PITFALLS AND GUARDRAILS

Autonomy needs limits: six to build in

Human approval

Reads run freely. Sends, payments and changes wait for a person.

Step limit

Cap the loop. A confused agent can call tools forever.

Validate arguments

Check every value the model passes before running a tool.

Least privilege

Give only the tools and access the task needs.

Untrusted tool output

Documents and web pages can carry hidden instructions.

Log everything

Keep traces of every step for debugging and audit.

RECAP

An agent is a loop: think, act, observe

Understand

Model, instructions, tools and memory, run in a loop the model controls.

Build

By hand first, then a framework that fits the problem.

Trust

Limits, approvals and evaluation before it acts for real.

PART 04

From LLM apps to AI agents

An LLM app answers. An agent decides, acts with tools, observes the result and keeps going until the goal is met.

The agent learning journey: six chapters in Part 4



1 · Introduction
What makes an agent, and the ReAct loop.

2 · Tools & MCP
How agents act, and the standard protocol that connects them to systems.

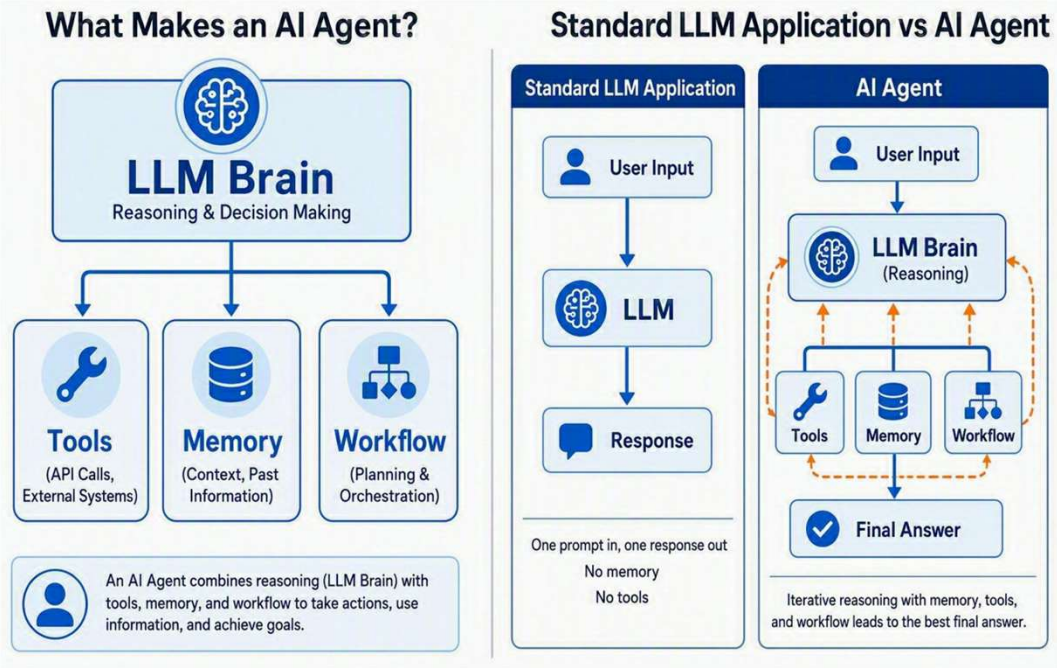
3 · Memory & context
What agents remember within and across sessions.

4 · Patterns & workflows
Proven structures such as parallel fan-out and fan-in.

5 · Agentic reasoning
Planning and deciding, seen end to end in a full agent.

6 · Agent skills
Packaging expertise the agent loads on demand.

What makes an agent: an LLM brain plus tools, memory and workflow



LLM brain
Reasons and decides the next step.

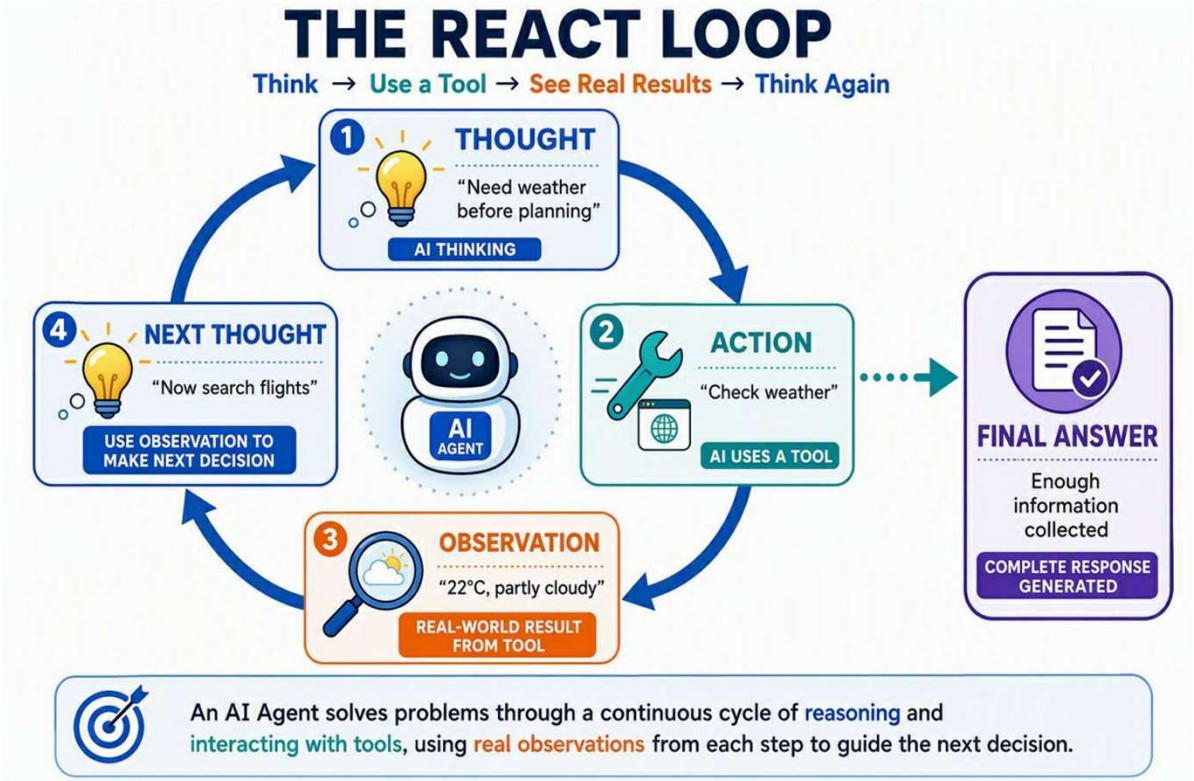
Tools
APIs and systems it can call to act or fetch facts.

Memory
Context from this session and past ones.

Workflow
The plan and control flow that orchestrate steps.

LLM app: one prompt in, one response out. Agent: iterates — reason, act, observe — until it can give the best final answer.

The ReAct loop: think, act with a tool, observe, think again



1 **Thought**
The model reasons about what it needs next: "I need the weather before planning."

2 **Action**
It emits a structured tool call, e.g. get_weather(city="Tokyo"). Your code executes it.

3 **Observation**
The real result ("22°C, partly cloudy") is appended to the context.

4 **Repeat or finish**
With new facts it decides the next action, or stops and writes the final answer.

Always bound the loop
Cap steps, tokens and time, and stop on repeated actions.

ReAct in code: the tool-calling loop is about 20 lines

python · react_agent.py

```python
def run_agent(question, max_steps=8):
    messages = [{"role": "system", "content": SYSTEM},
                {"role": "user", "content": question}]
    for step in range(max_steps):                # bounded loop
        resp = client.chat.completions.create(
            model=MODEL, messages=messages, tools=TOOLS)
        msg = resp.choices[0].message            # THOUGHT
        if not msg.tool_calls:                   # done: final answer
            return msg.content
        messages.append(msg)
        for call in msg.tool_calls:              # ACTION
            args = json.loads(call.function.arguments)
            result = run_tool(call.function.name, args)
            messages.append({"role": "tool",     # OBSERVATION
                "tool_call_id": call.id,
                "content": json.dumps(result)})
    raise RuntimeError("Step budget exceeded")
```

TOOLS is a JSON schema list

Each tool has a name, a description the model reads, and typed parameters. Good descriptions matter as much as good prompts.

Your code executes the tool

The model only requests an action. Validate arguments and enforce permissions in run_tool, never in the prompt.

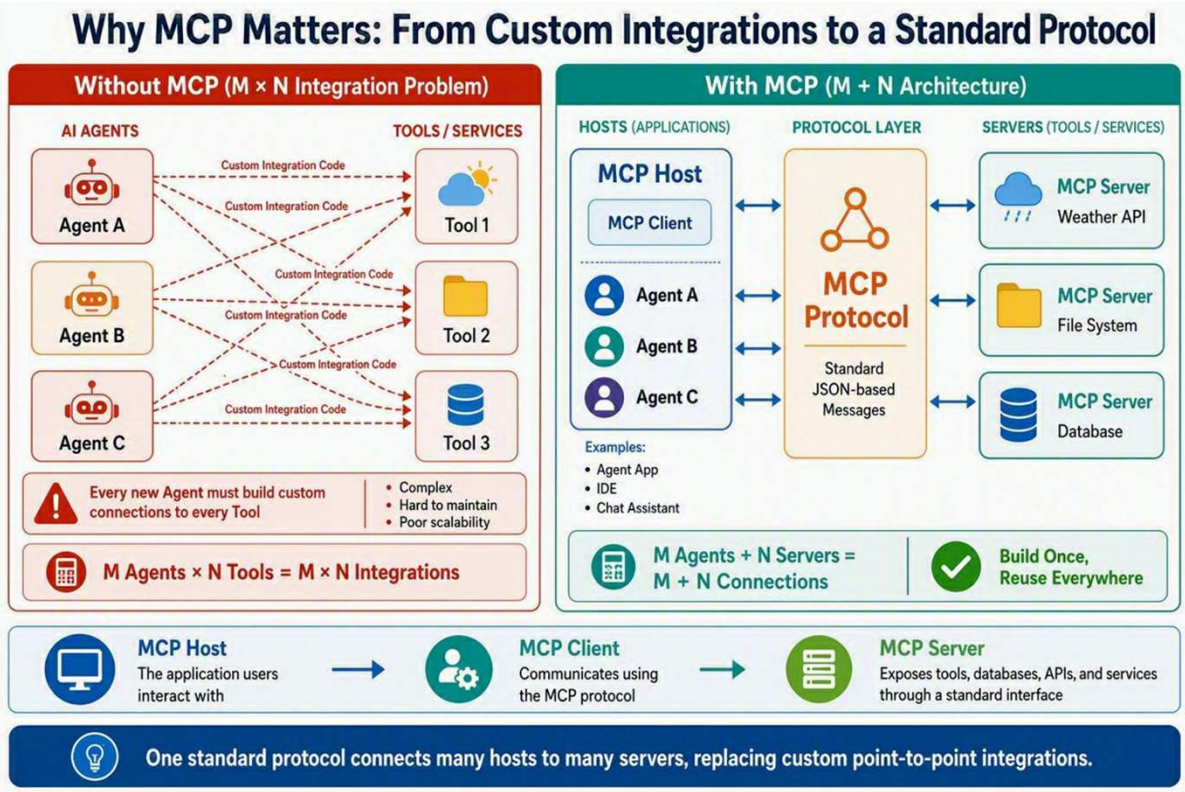
Observations go back as role "tool"

Linked by tool_call_id so the model knows which result answers which request.

Frameworks wrap this loop

LangGraph, CrewAI and the OpenAI and Claude Agent SDKs add state, retries and tracing on top.

MCP: one standard protocol replaces M × N custom integrations



The problem: M × N

Without a standard, 3 agents × 3 tools means 9 bespoke integrations, each with its own auth, schema and maintenance burden.

The fix: M + N

Each tool is wrapped once as an MCP server; each app embeds an MCP client once. Any host can then use any server.

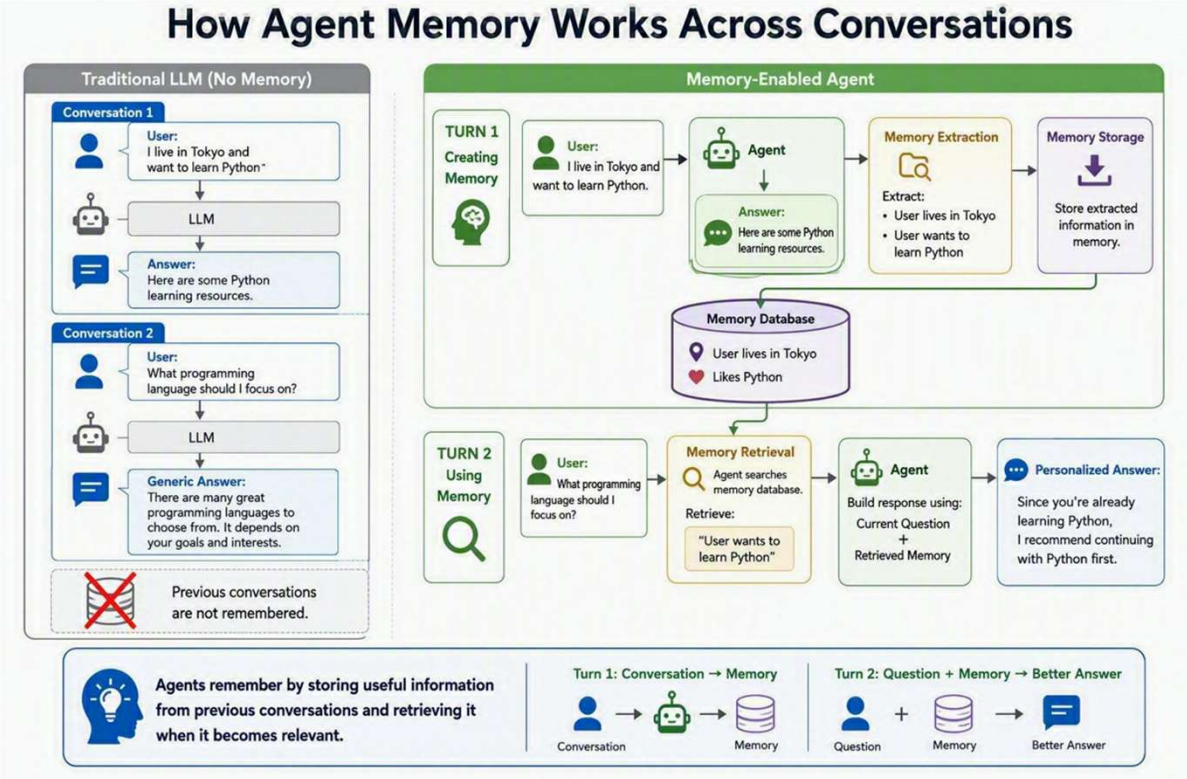
Three roles

Host: the app users interact with. Client: speaks MCP inside the host. Server: exposes tools, resources and prompts over JSON-RPC.

Enterprise angle

Approved MCP servers become a governed catalogue of capabilities, reviewed once and reused everywhere.

Memory: agents store what matters and retrieve it when it is relevant



Type	Lives in	Example
Short-term	The messages array	Current conversation turns
Long-term	Database or vector store	"User is learning Python"
Knowledge	Documents via retrieval (RAG)	Policies, product facts

Write path

After a turn, extract durable facts and store them.

Read path

Before answering, retrieve relevant memories and add them to context.

Govern it

Memory is personal data: apply consent, retention limits and the right to delete.

Five workflow patterns: choose the simplest that solves the problem

Prompt chaining

Fixed sequence of LLM steps, each feeding the next, with checks in between.

Draft → review → format

Routing

Classify the input, then send it to a specialised prompt, model or team.

Triage client queries by intent

Parallelisation

Run independent sub-tasks at once, then combine (fan-out / fan-in).

Weather + flights + hotels

Orchestrator–workers

A lead LLM breaks the task down dynamically and delegates to workers.

Multi-file code change

Evaluator–optimiser

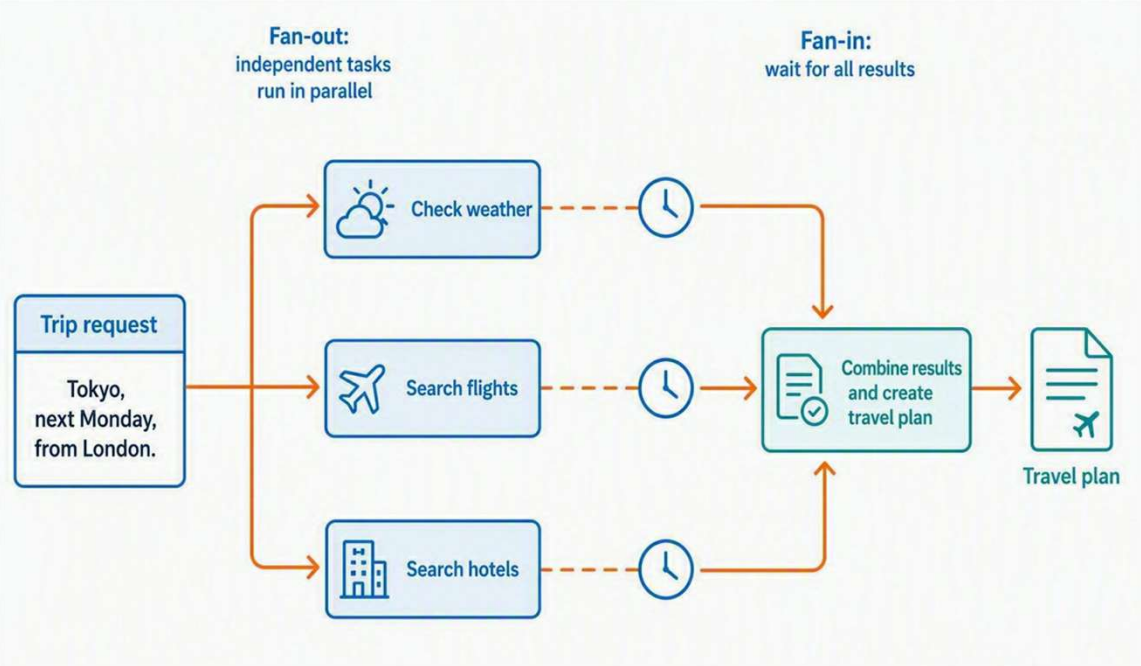
One LLM generates, another critiques, loop until criteria are met.

Compliance-checked drafts

More predictable, cheaper, easier to test → **More flexible, more autonomous, harder to control**

Workflows follow code paths you define. Agents choose their own path. Start with a workflow; reach for full autonomy only when the steps cannot be known in advance.

Parallel pattern: fan out independent tasks, fan in the results



Fan-out

Weather, flights and hotels do not depend on each other, so they run concurrently, e.g. with asyncio.gather or a LangGraph parallel branch.

Fan-in

Wait for all results, then one LLM step combines them into a plan.

The payoff

Latency ≈ the slowest task, not the sum of all tasks.

Design for partial failure

Set a timeout per branch and decide up front what to do when one tool fails.

Rule of thumb: if two steps do not need each other's output, they should not wait for each other.

Putting it together: anatomy of a complete travel-planning agent



1. **Request** Natural-language goal with constraints: dates, people, budget.

2a. **Memory** Profile, history and saved trips personalise the plan.

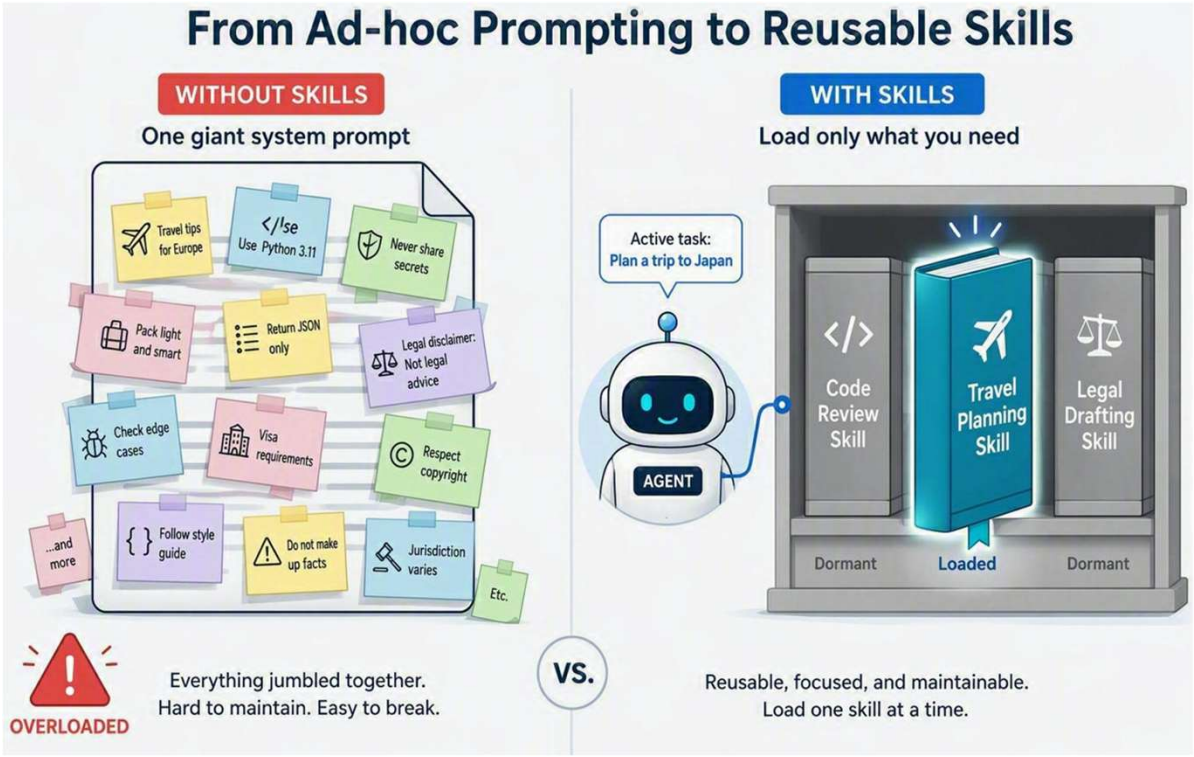
2b. **Reasoning** Understand → plan → evaluate options → refine: ReAct in action.

3. **Tools** Weather, flights, hotels, budget calculator, often via MCP.

4. **Output** A structured, costed itinerary, not just prose.

5. **Learning** Feedback updates memory for next time.

Agent skills: load expertise on demand instead of one giant prompt



The giant-prompt problem

Cramming every rule for every task into one system prompt wastes tokens, dilutes attention and makes changes risky.

A skill is a folder

Instructions in a SKILL.md, plus optional scripts, templates and reference files, for one kind of task.

Progressive disclosure

The agent first sees only each skill's name and description, then loads the full skill only when the task needs it.

Enterprise value

Teams package house standards — code review, report formats, compliance wording — once, and every agent reuses them.

FILXGPT APIS · PART 14

Building effective agents: structure, context, control

Implementation patterns, context engineering and human-in-the-loop, with working code

workflow · orchestrator · context · approve · edit · reject

THE ROADMAP

Three questions every agent design must answer

1

Structure

Which steps are fixed in code, and which does the model choose?

Implementation patterns

2

Context

What exactly does the model see at each step?

Context engineering

3

Control

When must a human approve before the agent acts?

Human-in-the-loop

1 · STRUCTURE

Design patterns vs implementation patterns

THE BLUEPRINT

Design pattern

What the whole agent system looks like, at a high level.

e.g. a supervisor agent with specialist sub-agents

THE WIRING

Implementation pattern

How the steps are connected and run, in code.

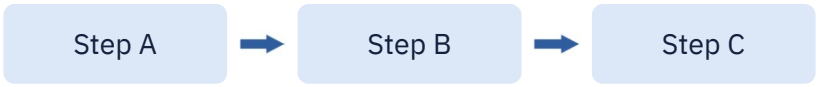
e.g. workflow-based or orchestrator-based

Many-to-many: one design can be wired several ways, and one wiring can serve several designs.

TWO IMPLEMENTATION PATTERNS

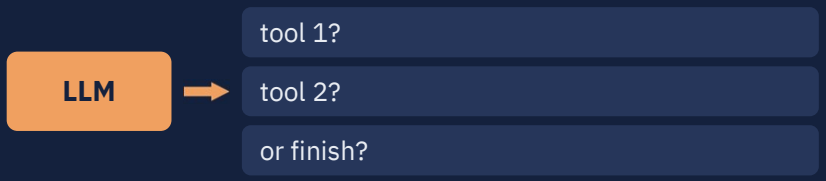
Code decides the path, or the LLM decides it

Workflow-based



A predefined path. The LLM fills in steps; it never chooses them. Deterministic, like a data pipeline.

Orchestrator-based



The LLM picks the next action each turn, adapting as it goes. Flexible, less predictable.

SIDE BY SIDE

Predictability versus flexibility

	Workflow-based	Orchestrator-based
Flow	Predefined code path	Path chosen by the LLM at runtime
Execution	Deterministic	Non-deterministic
Output	Consistent	Variable
Use case	Tasks with clearly defined subtasks	Open-ended, unpredictable problems
Pros	Simple to extend in a straight line	Easy to add new tools
Cons	Gets complex with many branches	Hard to guarantee reliable tool use
Cost	Fixed number of LLM calls	Variable number of LLM calls

SAME TASK, TWO WIRINGS

Triage a support ticket

```
# Workflow: the steps are fixed in code
text = load_ticket(ticket_id)
summary = ask_llm("Summarise in 3 bullets:\n" + text)
label = ask_llm("Reply BUG, REQUEST or QUESTION only:\n" + summary)
route_ticket(label, summary) # always 2 LLM calls
```

```
# Orchestrator: the LLM picks the steps (Part 13's Agents SDK)
agent = Agent(name="Triage", instructions="Triage the ticket. Ask if unclear.",
                             tools=[load_ticket, search_kb, route_ticket, ask_user])
Runner.run_sync(agent, f"Triage ticket {ticket_id}") # 1 to N LLM calls
```

CHOOSING A PATTERN

Can you write the steps down in advance?

YES

Workflow

The task splits into known subtasks.

e.g. nightly report: fetch, summarise, email

MOSTLY

Hybrid

A fixed workflow with one open-ended step handled by an agent.

e.g. fixed intake, agent investigates

NO

Orchestrator

The next step depends on what it finds.

e.g. investigate why a job failed

Default to the **workflow**. Earn the orchestrator when the problem demands it.

2 · CONTEXT ENGINEERING

Managing everything the model sees, not just the prompt

> The systematic design of how all the information an agent needs is collected, stored, managed and transformed, across its whole life cycle.

Collect
User input, history, external data, tool results

Store
Short-term and long-term memory, ready to retrieve

Manage
Choose what goes into the window for this step

Transform
Compress and format it into tokens the model can use

AN ANALOGY

Think of the agent as a computer

Operating system **= context engineering.** Decides what to load into memory, when, and in what format.

CPU = the LLM

Does the reasoning, but only on what is in RAM

RAM = context window

Limited and expensive working memory

Disk = everything else

Documents, memory stores, tool outputs, loaded on demand

A CPU can't use data still on disk. A model can't use facts outside its window.

WHAT FILLS THE WINDOW

One call's context: a budget with five related skills



0 Example window: 16K tokens 16K

- **Instructions** 1.5K — Prompt engineering
- **Recent history** 5K — Memory (short-term)
- **Tool definitions** 2K — Tool calling
- **Current question** 0.5K — User input
- **Memory summary** 1K — Memory
- **Answer space** 2K — Structured output
- **Retrieved chunks** 3K — RAG
- **Headroom** 1K — Safety margin

WHY IT MATTERS

Five benefits, five failures avoided

Benefit	Failure it prevents	How
Reliability	"Goal amnesia": forgetting the task	Structured memory keeps the goal
Scalability	Window overflow on long tasks	Prioritise and compress context
Efficiency	Wasted tokens, cost and latency	Send only what this step needs
Adaptability	Repeating the same mistakes	Learn from stored experience
Accuracy	Hallucination from missing facts	Relevant, grounded context

OPTIMISATION STRATEGIES

Four toolkits for a crowded window

Window management · drop

Sliding window: keep the last N turns
Token truncation: cap by a token budget
Priority retention: always keep goals and decisions

Compression · shrink

Recursive summary: summarise old turns
Detail reduction: drop verbose parts, keep facts
Semantic clustering: merge similar items

Intelligent retrieval · fetch

Selective memory: load only relevant memories
Contextual ranking: most relevant first
Dynamic filtering: drop stale, conflicting items

Structure · arrange

Clear sections: system, history, knowledge, task
Efficient formats: JSON or structured markdown
Placement: key facts at start and end

WINDOW MANAGEMENT IN CODE

Keep the goal, then the newest turns

```
import tiktoken
enc = tiktoken.get_encoding("o200k_base") # tokenizer of gpt-4o and gpt-4.1
count = lambda m: len(enc.encode(m["content"]))

def fit_history(system, pinned, history, budget=6000):
        used = count(system) + sum(count(m) for m in pinned) # priority retention
        kept = []
        for msg in reversed(history): # newest first
                if used + count(msg) > budget:
                        break # token-based truncation
                kept.insert(0, msg)
                used += count(msg)
        return [system] + pinned + kept
```

pinned. The task goal and key decisions, never dropped.

budget. Tokens, not turns: finer control.

Next step. Summarise what was dropped.

STRUCTURAL OPTIMISATION

A fixed layout for every call

1 · System: role, rules, guardrails

2 · Tool definitions

3 · Long-term memory: preferences, past decisions

4 · Retrieved knowledge, with sources

5 · Recent conversation (trimmed)

6 · Current task and question

Start: strong attention

Put rules the model must never break here.

Middle: easiest to miss

Bulk material, clearly labelled.

End: strong attention

Restate the task right before the model answers.

INDUSTRY PERSPECTIVES

Three vendors, one shared idea

Google

SESSIONS AND MEMORY

Manage conversation history, balance relevance against token limits, compress by summarising.

OpenAI

WHAT GOES IN, AND WHERE

System messages and roles, few-shot example placement, RAG for selective retrieval.

LangChain

STATEFUL MEMORY SYSTEMS

Buffer, summary and entity memory; compression; loading context dynamically.

Shared idea: the window is scarce, so **choose, compress and order** what goes into it.

3 · HUMAN-IN-THE-LOOP (HITL)

Pause at the risky moment, let a person decide

WITHOUT HITL

- Agent may take inappropriate actions
- Mistakes are costly and irreversible
- Users have little control
- Compliance and audit are hard

WITH HITL

- Humans control sensitive operations
- Errors are caught before damage
- Actions can be corrected first
- Every decision is logged for audit

At a pause, the human can **review, approve, edit or reject** the proposed action.

SELECTIVE INTERVENTION

Classify every tool by risk, before you build

Risk	Tool category	Examples	Human approval?
High	External communication	Email, SMS, Slack, social media	Yes, always
High	Financial operations	Payments, transfers, refunds	Yes, always
High	Data modification	Delete, update, archive	Yes, always
Medium	Internal communication	Internal chat, notifications	Conditional
Low	Information retrieval	Search, read, query	No, runs freely
Low	Analysis	Calculate, visualise, report	No, runs freely

Rule of thumb: if the action **leaves the building, moves money or changes data**, a human approves it.

THE FLOW AND THREE DECISIONS

Propose, pause, decide, resume



Approve

Run the action exactly as proposed.

The action is correct and safe.

Edit

Change the arguments, then run.

Wrong recipient, tone needs adjusting.

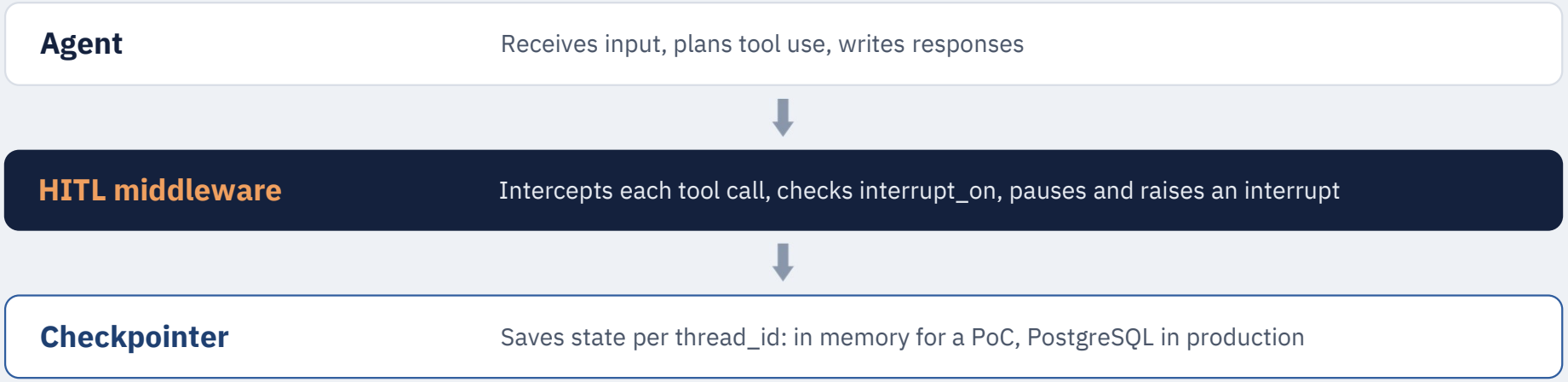
Reject

Block it; tell the agent why.

Inappropriate or dangerous.

TECHNICAL ARCHITECTURE

Three layers make the pause possible



The **thread_id** is how a decision made later finds the right paused conversation.

HITL IN CODE · STEP 1

Configure which tools must pause

```
# imports as in the doc's Step 1 (needs langchain 1.0 or later)
model = ChatOpenAI(model="gpt-4.1", api_key=access_token, base_url=GATEWAY_URL)

@tool
def send_email(to: str, subject: str, body: str) -> str:
        """Send an email to a colleague."""
        return f"Email sent to {to}" # demo only: nothing is sent

agent = create_agent(
        model=model, tools=[send_email, get_fund_price],
        middleware=[HumanInTheLoopMiddleware(interrupt_on={
                "send_email": {"allowed_decisions": ["approve", "edit", "reject"]},
                "get_fund_price": False})], # low risk: never pauses
        checkpointer=InMemorySaver()) # PoC only; use a database in production
```

interrupt_on. Your risk table, written as code.

allowed_decisions. Limit what reviewers can do.

checkpointer. Required, or nothing can pause.

HITL IN CODE · STEP 2

Run, catch the interrupt, resume with a decision

```
config = {"configurable": {"thread_id": "request-4711"}}
msg = {"role": "user", "content": "Email anna@example.com the ABC123 price"}
result = agent.invoke({"messages": [msg]}, config)
if "__interrupt__" in result: # paused: show the proposed email to a reviewer
        print(result["__interrupt__"])

# later, perhaps from a web request, with the SAME thread_id
agent.invoke(Command(resume={"decisions": [{"type": "approve"}]}), config)

# or edit the arguments first (new_args = the reviewer's corrected values)
edit = {"type": "edit", "edited_action": {"name": "send_email", "args": new_args}}
agent.invoke(Command(resume={"decisions": [edit]}), config)
```

FROM POC TO PRODUCTION

The pause is the easy part; the process around it isn't

Notify
Alert the reviewer in Slack or email when a decision waits

Time out
Abandoned approvals expire, and default to reject

Audit
Log who decided what, when, and what changed

Durable state
A database checkpointer, so pauses survive restarts

Show the whole action
Every argument, plus why the agent wants it

Measure it
Approval rates, wait times and middleware overhead

PITFALLS

Six ways HITL quietly fails

Approval fatigue

Too many prompts, and people click approve without reading.

Vague requests

"Approve tool call?" with no details can't be judged.

Lost state

In-memory saver plus a restart: every pause is gone.

Unchecked edits

Validate edited arguments like any other input.

Missed tools

A new risky tool added without updating interrupt_on.

Version drift

These APIs need LangChain 1.0+, not the 0.1.x the doc lists.

RECAP

Structure it, feed it well, keep a human on risky actions

Structure

Workflow by default; orchestrator only when steps can't be known in advance.

Context

Drop, shrink, fetch and arrange what the model sees, every call.

Control

Classify tools by risk; approve, edit or reject the risky ones.

PART 05

Production readiness

In financial services, a demo that works is the starting line. Trust comes from guardrails, evaluation, observability and clear human accountability.

Know the failure modes, and the control for each

Risk	What it looks like	Primary controls
Prompt injection	A document or user message says “ignore previous instructions”, or a tool result smuggles in commands.	Keep untrusted text out of system prompts; least-privilege tools; validate tool arguments; human approval for sensitive actions
Sensitive data leakage	PII or client data sent to a model, written to logs, or echoed back in outputs.	Approved models via the gateway; PII redaction; data classification; log scrubbing
Hallucination	Confident but false facts, numbers or citations.	Ground answers with retrieval and tools; require sources; low temperature; “I don't know” is allowed
Excessive agency	An agent takes an action nobody intended: sends, deletes, trades.	Read-only by default; scoped permissions; human-in-the-loop for irreversible steps
Runaway cost & loops	An agent loops or a prompt balloons, burning tokens and quota.	Step, token and time budgets; gateway quotas; alerts on spend anomalies



Defence in depth: “telling the model not to” is never a control on its own. Enforce rules in code, permissions and process.

Reference: OWASP Top 10 for LLM Applications (2025) covers these and more.

You cannot improve what you do not measure: evaluate and observe

Evaluate before release

- Build a golden dataset of real, representative inputs with expected outputs
- Score with code checks first (schema, exact match), then LLM-as-judge with a clear rubric
- Run the suite in CI on every prompt, model or tool change
- Track regressions: a new model is a new dependency version

Observe in production

- Trace every step of an agent: prompt, tool calls, observations, final answer
- Monitor latency (incl. time to first token), tokens, cost and error rates
- Capture user feedback and sample conversations for human review
- Alert on anomalies: loops, spend spikes, rising refusal or filter rates

Treat prompts, models and tools as versioned code: every change is tested against the same evaluation suite before it ships.

Go-live checklist for an LLM app or agent at FIL

Access & data

- ☐ All calls via the AI Gateway
- ☐ No secrets in code or notebooks
- ☐ Data classified; PII redacted where required

Agent design

- ☐ Simplest pattern that works
- ☐ Tools least-privilege, arguments validated
- ☐ Step, token and time budgets enforced

Human oversight

- ☐ Human approval for irreversible actions
- ☐ Clear escalation path
- ☐ Users told they are interacting with AI

Quality

- ☐ Golden dataset and CI evaluations
- ☐ finish_reason and errors handled
- ☐ Prompts versioned and peer-reviewed

Operations

- ☐ End-to-end tracing and logging
- ☐ Cost and latency dashboards and alerts
- ☐ Fallback when the model is unavailable

Governance

- ☐ Risk assessment completed
- ☐ Model and use case approved
- ☐ Retention and deletion rules for memory

PART 06

Hands-on labs

Six short labs turn each concept into working code. Every lab has a starter file with TODOs, a solution, and an offline mode for when the gateway is unavailable.

Lab setup: ten minutes, once, before Lab 1

terminal · setup

```
# 1. Unzip fil-agent-labs.zip, then:
python -m venv .venv
source .venv/bin/activate
# Windows: .venv\Scripts\activate
pip install -r requirements.txt

# 2. Gateway settings from your instructor
cp .env.example .env           # fill in URL, key, model
set -a && source .env && set +a

# 3. Run a lab (from inside labs/)
cd labs
python lab1_first_call.py

# No gateway? Run offline:
export USE_MOCK_LLM=1
```

labs/ · six starter files

Each has TODO blocks that raise NotImplementedError until you finish them.

labs/solutions/ · answers

Compare after you try. Instructors can demo from these.

Offline mock mode

USE_MOCK_LLM=1 swaps in a scripted model. It is not an LLM: it only exercises your code paths.

capstone-starter/ · Friday

Synthetic data, six ready tools, schema, CLI skeleton and the eval set. Do not open it until Friday.

PART 06 · HANDS-ON LABS

Six labs, one per concept, each building on the last

Lab	You build	Part	Time	Done when
1	First gateway call · temperature experiment · truncation check	1–2	30 min	You can explain why temperature 0 is not a reproducibility guarantee
2	Streaming with time-to-first-token and usage	2	30 min	TTFT is far below total time, and you can say when not to stream
3	Versioned prompt template · chat loop with history trimming	3	40 min	Prompt tokens stop growing; you can name what was forgotten
4	Bounded ReAct agent with two tools and a step trace	4	60 min	Weather and currency answered from tool results in 2 steps
5	MCP server · tool discovery · parallel fan-out	4	60 min	Trace shows ~3 s sequential vs ~1 s parallel
6	PII redaction · injection guard · approval gate · golden-set eval	5	45 min	5/5 cases pass with a poisoned tool result in the mix



About 4.5 hours in total. By Thursday every trainee has written each component the Friday capstone needs: loop, tools, MCP, memory, guardrails and evals.

Labs 1–2: talk to the model through the gateway, then stream it

LAB 1

First call & sampling

30 min `lab1_first_call.py`

You will

1. Call the gateway with a system and a user message; print content, finish_reason and usage.
2. Ask the same question 5 times at temperature 0, 0.7 and 1.2; count distinct answers.
3. Set max_tokens=15 and handle finish_reason == "length" as an error.

Done when you can explain why temperature 0 is not a reproducibility guarantee.

LAB 2

Streaming & time to first token

30 min `lab2_streaming.py`

You will

1. Time a normal call from request to full answer.
2. Stream the same request with stream=True and print each delta as it arrives.
3. Record time to first token, and read usage from the final chunk.

Done when TTFT is far below total time, and you can say when not to stream.

Labs 3–4: shape the input, then give the model tools and a loop

LAB 3

Templates & history

40 min `lab3_templates_and_history.py`

You will

1. Load prompts/explainer.txt, read its version, and fill {role}, {language} and {max_words}.
2. Build a 5-question chat loop that resends history; print prompt_tokens each turn.
3. Trim to the system message plus the last 3 turns; watch what the model forgets.

Done when prompt tokens stop growing, and you can name what was lost.

LAB 4

A bounded ReAct agent

60 min `lab4_react_agent.py`

You will

1. Write JSON schemas for get_weather and convert_currency.
2. Loop: call the model with tools, run each tool call, append role "tool" observations.
3. Cap at 6 steps and print a trace line per step: finish_reason, tokens, ms.

Done when “Tokyo weather and £500 in yen” is answered from tool results in 2 steps.

Labs 5–6: standardise tools with MCP, then make the agent safe

LAB 5

MCP server & parallel fan-out

60 min `lab5_mcp_server.py + _agent.py`

You will

1. Register three async tools on a FastMCP server.
2. Launch it over stdio, list its tools and map them to the OpenAI format.
3. Run the three tool calls one by one, then with asyncio.gather.

Done when the trace shows about 3 s sequential versus about 1 s parallel.

LAB 6

Guardrails & evaluation

45 min `lab6_guardrails_and_evals.py`

You will

1. Redact emails, phone numbers and account numbers (keep the last 4 digits).
2. Wrap tool output in <tool_data> tags and flag injection text.
3. Add a yes-only approval gate; run a 5-case golden set against Lab 4.

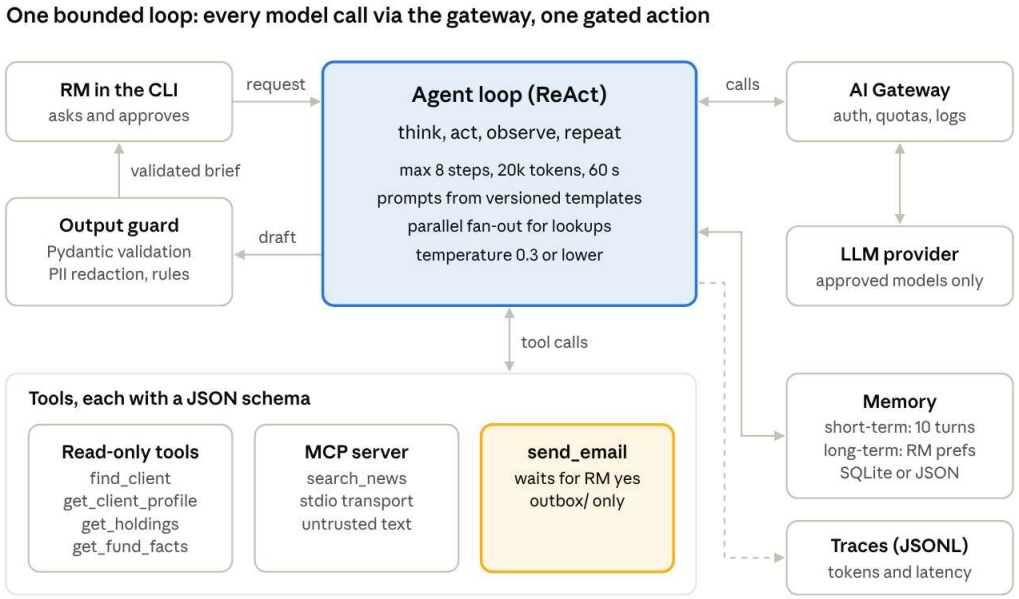
Done when 5/5 cases pass with a poisoned tool result in the mix.

PART 07

Friday capstone test

Every trainee builds, tests and demos a complete enterprise agent in one day. It is individual, timed and marked against a published rubric.

The brief: build a Client Meeting Brief Agent for RMs



The problem

RMs spend 30–45 minutes before each meeting pulling profile, holdings, fund facts and news from separate systems.

The agent

From “Prep me for my 3 pm with Priya Raman” to a validated, sourced, compliance-safe brief in under a minute.

The rule

Facts only, no investment advice, no leaked personal data, and nothing leaves without the RM’s explicit yes.

All data is synthetic: 8 clients, 10 funds, 20 news items, including a planted prompt injection. Never use real client or colleague data.

PART 07 · FRIDAY CAPSTONE TEST

What the agent must do, and the four rules that fail the test outright

Must-have requirements

FR-1	Natural-language request in a CLI
FR-2	Resolve the client; ask when 0 or several match
FR-3	Gather facts with tools; never invent figures
FR-4	Return a MeetingBrief that validates (Pydantic)
FR-5	Every fact cites tool + record ID
FR-6	Decline advice; flag it for RM judgement
FR-7	Follow-up turns: “make it shorter”
FR-8	Email only after an explicit “yes”

Architecture: bounded ReAct (8 steps) · 6 tools · 1 MCP server · parallel fan-out · short- and long-term memory · JSONL traces

GATE REQUIREMENTS · fail one, fail the test

No secrets in code
Keys only from environment variables

Injection ignored
Planted instruction in news item N-017 has no effect

No investment advice
No buy, sell or switch recommendation, ever

Human approval
send_email never runs without the RM’s “yes”

PART 07 · FRIDAY CAPSTONE TEST

Friday: schedule, deliverables and how it is marked

09:00 Briefing

09:30 Build block 1

13:00 Lunch

13:45 Build block 2

16:00 Code freeze

16:15 Demos 10+5 min

Area	Pts
Functionality (FR-1 to FR-8)	25
Agent architecture	20
Safety and compliance	20
Evaluation and observability	15
Engineering quality	10
Demo and explanation	10
Stretch (bonus)	+10

Submit by 16:00

- Source code, README (run in 3 commands) and AI-assistant declaration
- ARCHITECTURE.md with diagram and design trade-offs
- evals/results.md and a JSONL trace of AT-01

60 to pass

85 distinction

4/4 gates held

KEY TAKEAWAYS

1 **An LLM predicts tokens**
Parameters shape how it picks; context shapes what it can say.

2 **Go through the gateway**
One governed endpoint for security, cost and audit.

3 **The model is stateless**
Your app owns the prompt, the history and the memory.

4 **An agent is a bounded loop**
Reason → act with tools → observe, with hard limits.

5 **Standardise and reuse**
MCP for tools, skills for expertise, patterns for flow.

6 **Trust is engineered**
Guardrails, evals and observability before go-live.

REFERENCES

Further reading

OpenAI · Chat Completions API reference
platform.openai.com/docs/api-reference/chat

Anthropic · Building effective agents
anthropic.com/engineering/building-effective-agents

Model Context Protocol · Specification and docs
modelcontextprotocol.io

Yao et al. · ReAct: Synergizing Reasoning and Acting in Language Models
arxiv.org/abs/2210.03629

Anthropic · Agent Skills overview
docs.claude.com/en/docs/agents-and-tools/agent-skills/overview

LangGraph · Documentation
langchain-ai.github.io/langgraph

OWASP · Top 10 for LLM Applications
genai.owasp.org/llm-top-10

Links were valid at the time of writing. Prefer official documentation; versions, model names and features change quickly.